

TUGAS AKHIR

IMPLEMENTASI DAN EVALUASI TOOL REGISTRY

UNTUK

SKALABILITAS AI AGENT MULTI-MODUL PADA

PLATFORM

ERP RESTORAN ZERLO.ID

Sebagai Salah Satu Syarat untuk Memperoleh Gelar Sarjana
Komputer pada
Program Studi Sistem Informasi



Oleh :
220101010009, Muhammadridwan

PROGRAM STUDI PJJ SISTEM INFORMASI

UNIVERSITAS SIBER ASIA

Juni 2026

HALAMAN PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama : Muhammadridwan

NIM : 220101010009

Judul Tugas Akhir : IMPLEMENTASI DAN EVALUASI TOOL REGISTRY
UNTUK SKALABILITAS AI AGENT MULTI-MODUL PADA PLATFORM
ERP RESTORAN ZERLO.ID

Menyatakan dengan sesungguhnya bahwa Tugas Akhir ini merupakan hasil penelitian, pemikiran, dan pemaparan asli penulis sendiri. Penulis tidak mencantumkan tanpa pengakuan bahan-bahan yang telah dipublikasikan sebelumnya atau ditulis oleh orang lain, atau sebagai bahan yang pernah diajukan untuk gelar atau ijazah pada Universitas Siber Asia atau Perguruan Tinggi lainnya.

Apabila di kemudian hari terdapat penyimpangan dan ketidakbenaran dalam pernyataan ini, maka penulis bersedia menerima sanksi akademik sesuai dengan peraturan yang berlaku di Universitas Siber Asia.

Demikian pernyataan ini penulis buat.

Jakarta, ____/____/____

Yang membuat pernyataan,

(materai Rp10.000,-)

Muhammadridwan

HALAMAN PENGESAHAN

Tugas Akhir ini diajukan oleh:

Nama : Muhammadridwan

NIM : 220101010009

Program Studi : Sistem Informasi

Judul Tugas Akhir : IMPLEMENTASI DAN EVALUASI TOOL
REGISTRY UNTUK SKALABILITAS AI AGENT MULTI-MODUL
PADA PLATFORM ERP RESTORAN ZERLO.ID

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Komputer pada Program Studi Sistem Informasi Universitas Siber Asia.

Ketua Sidang / Dosen Penguji I

Dosen Penguji II

.....

.....

Dosen Pembimbing I

Dosen Pembimbing II

Ikhwani Saputra, S.Kom., M.Kom.

.....

Ketua Program Studi

.....

Ditetapkan di : Jakarta

Tanggal : ____/____/202__

ABSTRAK

Pertumbuhan katalog *tool* pada AI Agent multi-modul menimbulkan masalah *tool overload*: seluruh definisi *tool* dikirim ke LLM sehingga konsumsi token meningkat proporsional terhadap ukuran katalog dan akurasi menurun akibat *context rot*. Penelitian ini merancang, mengimplementasikan, dan mengevaluasi Tool Registry deterministik berbasis metadata sebagai lapisan penyaringan *tool* sebelum inferensi pada platform ERP restoran zerlo.id. Registry menyaring *tool* berdasarkan modul, peran, tingkat langganan, kata kunci, dan anggaran token dengan batas 15 *tool* per panggilan. Evaluasi menggunakan pendekatan *Design Science Research* dengan 100 kueri berbahasa Indonesia, tiga pengulangan, dan tiga skenario katalog (30, 100, dan 300 *tool*) pada Gemini 2.5 Flash Lite, total 558 rekaman, diuji dengan Wilcoxon *signed-rank*, Cohen's d, dan *confidence interval* 95%. Hasil menunjukkan penghematan token 63–95% yang signifikan secara statistik ($p < 0,001$; Cohen's d 11,7–782) dengan jumlah *tool* terlihat tetap konstan pada 15 meskipun katalog tumbuh sepuluh kali lipat, membuktikan skalabilitas *sub-linear* $O(1)$. Akurasi pemilihan *tool* meningkat +6,3 poin persen sebagai tren *directional*. Tool Registry memungkinkan AI Agent enterprise diskalakan ke ratusan modul tanpa lonjakan biaya inferensi maupun degradasi konteks.

Kata Kunci : AI Agent, efisiensi token, function calling, skalabilitas, tool registry

ABSTRACT

The growth of tool catalogs in multi-module AI Agents causes *tool overload*: all tool definitions are sent to the LLM, so token consumption scales proportionally with catalog size and accuracy degrades due to *context rot*. This research designs, implements, and evaluates a deterministic metadata-based Tool Registry as a tool-filtering layer prior to inference on the zerlo.id restaurant ERP platform. The registry filters tools by module, role, subscription tier, keywords, and token budget, capped at 15 tools per call. The evaluation follows the *Design Science Research* approach using 100 Indonesian-language queries, three repeats, and three catalog scenarios (30, 100, and 300 tools) on Gemini 2.5 Flash Lite, totaling 558 records, tested with the Wilcoxon *signed-rank* test, Cohen's d, and 95% *confidence intervals*. The results show a statistically significant token reduction of 63–95% ($p < 0.001$; Cohen's d 11.7–782), while the number of visible tools remains constant at 15 even as the catalog grows tenfold, demonstrating *sub-linear* $O(1)$ scalability. Tool selection accuracy improves by +6.3 percentage points as a *directional* trend. The Tool Registry enables enterprise AI Agents to scale to hundreds of modules without proportional inference-cost growth or context degradation.

Keywords : AI Agent, function calling, scalability, token efficiency, tool registry

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya penulis dapat menyelesaikan Tugas Akhir ini. Penulisan Tugas Akhir ini dilakukan dalam rangka memenuhi salah satu syarat untuk mencapai gelar Sarjana Komputer pada Program Studi Sistem Informasi Universitas Siber Asia.

Penulis menyadari bahwa tanpa bantuan dan bimbingan dari berbagai pihak, dari masa perkuliahan sampai pada penyusunan Tugas Akhir ini, sangatlah sulit bagi penulis untuk menyelesaikan Tugas Akhir ini. Oleh karena itu, penulis mengucapkan terima kasih kepada:

1. Ikhwani Saputra, S.Kom., M.Kom., selaku dosen pembimbing yang telah menyediakan waktu, tenaga, dan pikiran untuk mengarahkan penulis dalam penyusunan Tugas Akhir ini;
2. tim dan komunitas pengembang platform zerlo.id yang telah banyak membantu dalam penyediaan data dan lingkungan eksperimen;
3. orang tua dan keluarga penulis yang telah memberikan bantuan dukungan material dan moral; dan
4. sahabat yang telah banyak membantu penulis dalam menyelesaikan Tugas Akhir ini.

Akhir kata, penulis berharap Tuhan Yang Maha Esa berkenan membalas segala kebaikan semua pihak yang telah membantu. Semoga Tugas Akhir ini membawa manfaat bagi pengembangan ilmu.

Jakarta, Juni 2026

Penulis

DAFTAR ISI

HALAMAN PERNYATAAN ORISINALITAS.....	ii
HALAMAN PENGESAHAN.....	iii
ABSTRAK.....	iv
ABSTRACT.....	iv
KATA PENGANTAR.....	v
DAFTAR ISI.....	vi
DAFTAR TABEL.....	viii
DAFTAR GAMBAR.....	ix
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Rumusan Masalah.....	3
1.3 Batasan Masalah.....	4
1.4 Tujuan Penelitian.....	4
1.5 Manfaat Penelitian.....	5
1.6 Metode Penelitian.....	5
BAB II LANDASAN TEORI.....	7
2.1 LLM dan Pemanggilan Fungsi (Tool-Calling).....	7
2.2 Rekayasa Konteks dan Context Rot.....	8
2.3 Tool RAG — Retrieval Tool Berbasis Vektor.....	9
2.4 Tool Registry dan Manajemen Tool.....	10
2.5 Antarmuka Native Pydantic AI vs Tool Registry zerlo.id.....	12
2.6 Design Science Research.....	12
2.7 Penelitian Terdahulu.....	14
BAB III IMPLEMENTASI METODE USULAN.....	16
3.1 Analisis Sistem Eksisting.....	16
3.2 Perancangan Tool Registry.....	17
3.3 Perancangan Eval Framework.....	18
3.4 Implementasi Tool Registry.....	21
3.5 Implementasi Eval Runner.....	22
3.6 Setup Eksperimen.....	23

BAB IV HASIL DAN ANALISA.....	25
4.1 Hasil Benchmark Synthetic S1–S4.....	25
4.2 Hasil Gemini Native v2 (Hasil Resmi).....	26
4.3 Analisis Token Reduction.....	26
4.4 Analisis Tool Selection Accuracy.....	27
4.5 Analisis Sub-linear Scalability.....	28
4.6 Analisis Memory Footprint.....	29
4.7 Analisis Latency.....	30
4.8 Uji Statistik.....	31
4.9 Failure Analysis.....	32
4.10 Validasi Multi-model.....	32
BAB V KESIMPULAN.....	34
5.1 Kesimpulan.....	34
5.2 Keterbatasan dan Ancaman Terhadap Validitas.....	36
5.3 Saran / Future Work.....	37
DAFTAR PUSTAKA.....	39
LAMPIRAN.....	41
Lampiran A: Source Code Tool Registry (Cuplikan Kunci).....	41
Lampiran B: Dataset 100 Kueri Evaluasi (JSON).....	42
Lampiran C: Hasil Mentah Eksperimen (CSV).....	43
Lampiran D: Script Analisis Statistik (Python).....	44
Lampiran E: Bukti Similarity Check Turnitin ($\leq 30\%$).....	48

DAFTAR TABEL

Tabel 2.1 Perbandingan Penelitian Terdahulu.....	14
Tabel 3.1 Atribut Metadata Tool.....	17
Tabel 3.2 Definisi Skenario Eksperimen.....	18
Tabel 3.3 Komposisi Dataset 100 Kueri.....	19
Tabel 3.4 Parameter Setup Eksperimen.....	23
Tabel 4.1 Hasil Benchmark Synthetic Token S1–S4.....	25
Tabel 4.2 Ringkasan Hasil Resmi Gemini Native v2.....	26
Tabel 4.3 Penghematan Token Registry terhadap Baseline.....	27
Tabel 4.4 Akurasi Pemilihan Tool per Skenario.....	28
Tabel 4.5 Jumlah Tool Terlihat dan Kompleksitas Penskalaan.....	29
Tabel 4.6 Jejak Memori Registry per Skenario.....	29
Tabel 4.7 Latensi p50 dan p95 per Skenario.....	30
Tabel 4.8 Hasil Uji Statistik Penghematan Token.....	31
Tabel 4.9 Kueri yang Gagal Konsisten di Seluruh Pengulangan.....	32
Tabel 5.1 Ancaman Terhadap Validitas, Mitigasi, dan Penelitian Lanjutan.....	37

DAFTAR GAMBAR

Gambar 2.1 Tahapan Design Science Research pada Penelitian Ini.....	13
Gambar 3.1 Arsitektur Pipeline Penyaringan Tool Registry.....	18
Gambar 3.2 Alur Kerja Kerangka Evaluasi Baseline dan Registry.....	20
Gambar 4.1 Perbandingan Rata-rata Token per Kueri Baseline dan Registry.....	26
Gambar 4.2 Perbandingan Akurasi Pemilihan Tool Baseline dan Registry.....	27
Gambar 4.3 Jumlah Tool Terlihat oleh LLM terhadap Ukuran Katalog.....	28
Gambar 4.4 Jejak Memori Registry terhadap Ukuran Katalog.....	29
Gambar 4.5 Perbandingan Latensi p50 Baseline dan Registry.....	30

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan kecerdasan buatan (*artificial intelligence*) pada dekade terakhir telah mendorong kemunculan sistem AI Agent yang mampu memanggil fungsi eksternal secara otonom untuk menyelesaikan tugas kompleks. Kemampuan ini, yang dikenal sebagai *function calling* atau *tool use*, pertama kali diperkenalkan sebagai fitur siap produksi oleh OpenAI pada Juni 2023 [1] dan kemudian diadopsi oleh Google DeepMind melalui mekanisme *FunctionDeclaration* pada Gemini [2]. Fondasi teoritis dari kemampuan ini diletakkan oleh Schick et al. (2023) melalui Toolformer [3], yang membuktikan bahwa LLM dapat belajar memanggil *tools* secara *self-supervised*, serta oleh Yao et al. (2023) melalui kerangka ReAct [4] yang memodelkan siklus Thought→Action→Observation sebagai dasar perilaku agen.

Zerlo.id adalah platform ERP (*Enterprise Resource Planning*) berbasis AI yang dirancang khusus untuk usaha mikro, kecil, dan menengah (UMKM) di sektor F&B (*Food and Beverage*) Indonesia. Platform ini mengoperasikan 38 modul bisnis dengan 1.176 *endpoint*, 11 AI Agent aktif, dan lebih dari 60 *tools* produksi yang tersedia bagi agen. Setiap modul — mulai dari manajemen pemesanan, inventaris, akuntansi, hingga analitik penjualan — memiliki sejumlah *tools* yang harus dapat dipanggil oleh AI Agent sesuai konteks permintaan pengguna. Pada tahap *beta testing* saat ini, jumlah *tools* terus bertambah sejalan dengan pengembangan fitur baru.

Pertumbuhan katalog *tools* ini menimbulkan permasalahan fundamental yang disebut *tool overload*. Ketika seluruh daftar *tools* dikirimkan kepada LLM sebagai bagian dari konteks (*context window*), jumlah token input meningkat secara proporsional terhadap ukuran katalog. Pada skenario S3 dengan 300 *tools*, pengukuran pada penelitian ini menunjukkan rata-rata konsumsi token mencapai 23.893 token per kueri hanya untuk mendefinisikan *tools* yang tersedia — bahkan sebelum pertanyaan pengguna diproses. Kondisi ini berdampak langsung pada biaya komputasi, latensi respons, dan akurasi pemilihan *tool*.

Fenomena degradasi performa LLM akibat konteks yang terlalu panjang telah dibuktikan secara empiris oleh Liu et al. (2024) dalam penelitian "Lost in the Middle" [5]: model cenderung mengabaikan informasi yang berada di bagian tengah konteks panjang dan lebih mengandalkan informasi di awal atau akhir konteks. Temuan serupa dilaporkan oleh Timothy B. Lee (2025) [6], yang mengukur akurasi Claude 3.5 Sonnet menurun dari 88% menjadi 30% ketika konteks mencapai 32K token — sebuah fenomena yang disebut *context rot*. Rajasekaran et al. (2025) dari Anthropic memperkuat temuan ini dengan pernyataan bahwa "*bloated tool sets cause agent failure*" dan merekomendasikan prinsip *minimal token set* [7].

Solusi yang berkembang di industri dan komunitas riset adalah pendekatan *Tool RAG (Retrieval-Augmented Generation untuk tools)*: alih-alih menyertakan seluruh katalog, sistem mengambil hanya *tools* yang paling relevan secara semantik sebelum memanggil LLM. Gorilla [8] dan ToolLLM [9] membuktikan bahwa LLM dapat dihubungkan dengan ribuan API nyata menggunakan mekanisme *retrieval*. Lumer (2024) melalui Toolshed [10] mencapai peningkatan Recall@5 sebesar 46–56% menggunakan fusi tiga fase RAG-*tool*. Namun pendekatan berbasis *embedding* ini membawa kompleksitas infrastruktur tersendiri: diperlukan model *embedding*, basis data vektor, dan *pipeline* sinkronisasi yang tidak selalu tersedia dalam konteks sistem ERP berukuran menengah.

Sebagai alternatif yang lebih deterministik dan ringan, konsep *Tool Registry* menawarkan lapisan penyaringan berbasis metadata: setiap *tool* didaftarkan bersama atribut modul, peran (*role*), tingkat langganan (*subscription tier*), kata kunci (*keywords*), dan batas anggaran token (*token budget*). Sebelum LLM dipanggil, registry memfilter katalog berdasarkan konteks permintaan sehingga hanya subset *tools* yang relevan yang diteruskan. Gaurav et al. (2025) menunjukkan bahwa pemuatan *tool* yang dinamis dapat mengurangi beban pemuatan *tool* hingga 50% sambil mempertahankan akurasi [11]. Jia dan Li (AAAI 2026) melalui AutoTool [12] membuktikan bahwa pola penggunaan *tools* bersifat sekuensial dan dapat diprediksi, mendukung preseleksi sebelum inferensi. Wang et al. (ACL 2026) melalui MCP-Flow [13] melaporkan validasi skala

industri dengan 11.536 *tools* di 1.166 server MCP. Di tingkat *hyperscaler*, AWS telah mengadopsi pola ini sebagai AWS Agent Registry dengan *hybrid search* dan integrasi IAM [14].

Pada ekosistem Pydantic AI — kerangka AI Agent yang digunakan zerlo.id — tersedia antarmuka `FilteredToolset` yang memungkinkan penyaringan *tools* berdasarkan predikat tunggal. Namun antarmuka ini tidak mendukung penyaringan multi-kriteria (modul + peran + tier + anggaran token) yang diperlukan oleh platform ERP multi-modul. Kesenjangan ini menjadi motivasi utama perancangan Tool Registry khusus yang menjadi objek penelitian ini.

Berdasarkan pemaparan di atas, penelitian ini membangun dan mengevaluasi Tool Registry deterministik berbasis metadata pada platform zerlo.id, dengan mengukur dampaknya terhadap efisiensi token, akurasi pemilihan *tool*, dan skalabilitas AI Agent secara kuantitatif. Sejauh pengetahuan penulis, belum ada penelitian yang secara eksplisit mengukur dan mempublikasikan hasil evaluasi kuantitatif Tool Registry berbasis metadata pada kerangka Pydantic AI 1.x.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, rumusan masalah penelitian ini dinyatakan sebagai berikut.

1. Diperlukan rancangan Tool Registry yang mampu menyimpan metadata *tool* secara terstruktur pada platform ERP multi-modul zerlo.id.
2. Diperlukan mekanisme penyaringan *tool* (*filtering*) secara dinamis berdasarkan modul, peran, tingkat langganan, dan anggaran token pada AI Agent.
3. Dampak penerapan Tool Registry terhadap penggunaan token (*token usage*), latensi (*latency*), dan akurasi pemilihan *tool* (*tool selection accuracy*) dibandingkan dengan pendekatan *baseline* tanpa penyaringan belum diketahui dan perlu diukur secara kuantitatif.

4. Kemampuan Tool Registry dalam mendukung skalabilitas AI Agent pada platform ERP restoran ketika jumlah *tools* dalam katalog bertambah secara signifikan perlu dibuktikan.

1.3 Batasan Masalah

Agar penelitian ini terfokus dan hasilnya dapat diukur secara valid, ditetapkan batasan-batasan sebagai berikut.

1. Penyedia LLM yang digunakan adalah Google Gemini 2.5 Flash Lite melalui Google Gen AI SDK dengan mekanisme *native function calling* (FunctionDeclaration, mode=ANY, temperature=0). Eksperimen multi-model (MiniMax-M2.7 dan Claude Sonnet 4.6) disertakan sebagai bukti generalisabilitas, bukan sebagai hasil utama.
2. Dataset evaluasi terdiri dari 100 kueri berbahasa Indonesia yang mencakup tiga kategori: 50 kueri *single-domain*, 30 kueri *cross-domain*, dan 20 kueri adversarial. Setiap kueri diulang sebanyak tiga kali per skenario (*repeat runs*).
3. Evaluasi dilakukan pada tiga skenario katalog: S1 (30 *tools*), S2 (100 *tools*), dan S3 (300 *tools*). Skenario S4 (1.000 *tools*) hanya disimulasikan secara deterministik, tidak dijalankan secara *live*.
4. Batas *tool budget* yang diteruskan ke LLM ditetapkan maksimal 15 *tools* per panggilan registry.
5. Penelitian tidak mengukur kualitas linguistik dari respons yang dihasilkan LLM.
6. Pendekatan *Tool RAG* berbasis *embedding* dibahas sebagai *future work*, bukan sebagai bagian dari implementasi utama.

1.4 Tujuan Penelitian

Penelitian ini bertujuan untuk:

1. merancang Tool Registry dengan metadata terstruktur (ToolMeta) yang mencakup atribut modul, tipe operasi, kata kunci, peran, dan tingkat langganan;

2. mengimplementasikan mekanisme penyaringan dinamis berbasis multi-kriteria (modul, peran, tier, token budget) sebagai lapisan antara katalog *tools* dan LLM pada AI Agent zerlo.id;
3. mengukur dampak penerapan Tool Registry terhadap penggunaan token, latensi, dan akurasi pemilihan *tool* secara kuantitatif menggunakan uji statistik Wilcoxon *signed-rank*, Cohen's d, dan *confidence interval* 95%;
4. membuktikan sifat skalabilitas *sub-linear* Tool Registry — yaitu bahwa jumlah *tools* yang terlihat oleh LLM tetap konstan ($O(1)$ terhadap budget) meskipun ukuran katalog bertambah dari 30 ke 100 ke 300 *tools*.

1.5 Manfaat Penelitian

Penelitian ini memberikan manfaat pada tiga dimensi berikut.

Manfaat Akademik. Penelitian ini menghasilkan *baseline* kuantitatif pertama evaluasi Tool Registry berbasis metadata pada kerangka Pydantic AI 1.x. Metodologi eksperimen — 100 kueri $\times$ 3 pengulangan $\times$ 3 skenario dengan uji Wilcoxon dan Cohen's d — dapat dijadikan acuan bagi penelitian serupa yang membandingkan strategi penyaringan *tool* pada sistem AI Agent enterprise.

Manfaat Praktis. Bagi platform zerlo.id, hasil penelitian ini memberikan bukti kuantitatif bahwa Tool Registry mampu menekan konsumsi token hingga 95% (skenario S3) tanpa degradasi akurasi, sehingga platform dapat diskalakan ke ratusan modul tanpa mengalami *context rot* atau lonjakan biaya inferensi yang proporsional.

Manfaat bagi Komunitas Pengembang. Pola arsitektur Tool Registry deterministik berbasis metadata yang dikembangkan dalam penelitian ini dapat direplikasi oleh pengembang sistem AI Agent enterprise lain yang memerlukan solusi *tool management* tanpa ketergantungan pada infrastruktur *embedding* dan basis data vektor.

1.6 Metode Penelitian

Penelitian ini menggunakan pendekatan *Design Science Research* (DSR) sebagaimana dikemukakan oleh Hevner et al. (2004) [15], yang terdiri dari lima tahap: identifikasi masalah (*problem identification*), penetapan tujuan solusi

(*objective of solution*), perancangan dan pengembangan artefak (*design and development*), demonstrasi (*demonstration*), dan evaluasi (*evaluation*).

Tahap Identifikasi Masalah. Fenomena *tool overload* diidentifikasi dari data operasional zerlo.id: 60+ *tools* aktif di lingkungan produksi dengan proyeksi pertumbuhan sejalan ekspansi modul. Literatur empiris ("Lost in the Middle" [5], *context rot* [6]) dikutip sebagai justifikasi akademik.

Tahap Perancangan Artefak. Artefak utama yang dirancang adalah: (a) *dataclass* ToolMeta sebagai skema metadata *tool*, (b) kelas ToolRegistry sebagai *singleton* penyimpan katalog, dan (c) fungsi `registry_filter()` sebagai *pipeline* penyaringan berbasis *keyword scoring* dan *budget cap*. Artefak pendukung mencakup *framework* evaluasi: *runner* JSONL inkremental, modul pengukuran statistik, dan *report generator*.

Tahap Demonstrasi. Tool Registry diintegrasikan ke dalam AI Agent berbasis Pydantic AI dan diuji pada tiga skenario katalog (S1/S2/S3) menggunakan dataset 100 kueri berbahasa Indonesia.

Tahap Evaluasi. Hasil eksperimen dianalisis menggunakan:

- (a) pengujian statistik Wilcoxon *signed-rank* berpasangan (satu arah) untuk menguji signifikansi penurunan token;
- (b) Cohen's d untuk mengukur besar efek (*effect size*);
- (c) *confidence interval* 95% untuk estimasi rentang penurunan;
- (d) analisis perbandingan akurasi pemilihan *tool* antara mode *baseline* dan *registry*.

Seluruh data eksperimen disimpan dalam format JSONL yang dapat dilanjutkan (*resume-safe*) untuk memastikan reproduktibilitas. Total rekaman yang dihasilkan adalah $n=558$ (279 *baseline* + 279 *registry*) dari eksperimen resmi Gemini Native v2.

BAB II LANDASAN TEORI

Bab ini menguraikan landasan teori dan tinjauan pustaka yang menjadi dasar penelitian. Pembahasan dimulai dari konsep dasar pemanggilan fungsi oleh *Large Language Model* (LLM), fenomena degradasi performa akibat konteks panjang, pendekatan *Tool RAG* dan *Tool Registry* sebagai solusi, posisi penelitian terhadap antarmuka *native* kerangka Pydantic AI, serta kerangka metodologis *Design Science Research*. Bab ditutup dengan tabel perbandingan penelitian terdahulu untuk menegaskan kebaruan penelitian ini.

2.1 LLM dan Pemanggilan Fungsi (Tool-Calling)

Kemampuan LLM untuk memanggil fungsi eksternal merupakan fondasi dari sistem AI Agent modern. Schick et al. (2023) melalui Toolformer [3] membuktikan bahwa LLM dapat belajar memutuskan *tool* mana yang dipanggil, kapan memanggilnya, argumen apa yang diberikan, serta bagaimana mengintegrasikan hasilnya ke dalam prediksi token berikutnya — seluruhnya secara *self-supervised* tanpa anotasi manusia dalam skala besar. Temuan ini mengatasi keterbatasan fundamental LLM dalam aritmetika dan pencarian fakta, sekaligus membuktikan bahwa kapabilitas *tool use* dapat ditanamkan ke dalam LLM dengan *overhead* pelatihan yang minimal.

Kemampuan tersebut dioperasionalkan ke dalam sebuah arsitektur agen melalui kerangka ReAct yang diperkenalkan oleh Yao et al. (2023) [4]. ReAct menggabungkan penalaran berantai (*chain-of-thought reasoning*) dengan aksi pemanggilan *tool* secara berselang-seling melalui siklus Thought→Action→Observation. Model menyusun rencana (*Thought*), memanggil sebuah *tool* (*Action*), mengamati hasilnya (*Observation*), lalu memperbarui rencana pada iterasi berikutnya. Pada *benchmark* ALFWorld dan WebShop, ReAct mengungguli pendekatan *imitation learning* dan *reinforcement learning* dengan margin absolut masing-masing 34% dan 10%. Kerangka ini menjadi rujukan standar bagi arsitektur AI Agent modern, termasuk agen yang dibangun pada platform zerlo.id.

Secara industri, kemampuan *tool-calling* dijadikan fitur siap produksi pertama kali oleh OpenAI pada Juni 2023 [1]. Pengumuman tersebut

memungkinkan pengembang mendefinisikan fungsi dan memperoleh keluaran JSON terstruktur berisi argumen pemanggilan fungsi dari model yang telah di-*fine-tune* untuk mendeteksi kapan sebuah fungsi perlu dipanggil. Standar *de facto* ini kemudian diadopsi oleh Anthropic dan Google. Google DeepMind menyediakan mekanisme *native function calling* pada Gemini melalui *FunctionDeclaration* [2], di mana skema *tool* didefinisikan sebagai JSON Schema yang diteruskan ke model sebelum inferensi, dan model memutuskan *tool* yang dipanggil berdasarkan konteks percakapan. Penelitian ini menggunakan mekanisme *native function calling* Gemini 2.5 Flash Lite sebagai *backend* pengukuran, sehingga pemahaman atas cara *tool* didefinisikan dan dipilih oleh model menjadi fondasi teknis langsung bagi evaluasi Tool Registry.

2.2 Rekayasa Konteks dan Context Rot

Penyertaan seluruh definisi *tool* ke dalam jendela konteks (*context window*) LLM menimbulkan konsekuensi yang telah dibuktikan secara empiris. Liu et al. (2024) dalam penelitian "Lost in the Middle" [5] menemukan bahwa performa LLM optimal ketika informasi relevan berada di awal atau akhir konteks, namun menurun signifikan ketika informasi tersebut berada di bagian tengah konteks panjang. Efek ini muncul bahkan pada model yang secara eksplisit dirancang untuk konteks panjang. Implikasinya langsung bagi penelitian ini: pemberian 300 definisi *tool* sekaligus kepada LLM — sebagaimana terjadi pada skenario S3 *baseline* yang mengonsumsi 23.893 token — secara inheren menurunkan akurasi pemilihan *tool* karena sebagian definisi pasti berada di posisi tengah yang rawan diabaikan.

Fenomena degradasi ini dikenal sebagai *context rot*. Lee (2025) [16] mendokumentasikan bahwa akurasi Claude 3.5 Sonnet menurun dari 88% menjadi 30% pada penalaran *two-hop* ketika konteks diperpanjang hingga 32.000 token, dan menjelaskan akar arsitekturalnya: mekanisme *attention* memiliki sifat penskalaan kuadratik ($O(n^2)$) sehingga setiap token tambahan meningkatkan beban komputasi dan menurunkan kemampuan model merecall informasi secara akurat. Penelitian lanjutan [17] memperkuat temuan ini dengan memisahkan efek panjang konteks dari kualitas *retrieval* — dengan mengendalikan *perfect retrieval*

(informasi relevan selalu tersedia) dan hanya memvariasikan panjang konteks total, degradasi performa tetap terjadi semata-mata karena konteks lebih panjang. Temuan ini membantah anggapan bahwa jendela konteks yang besar sudah memadai.

Sebagai respons, Rajasekaran et al. (2025) dari Anthropic [7] mendefinisikan *context engineering* sebagai strategi mengkurasi dan mempertahankan himpunan token optimal selama inferensi LLM. Tiga prinsip yang dikemukakan relevan langsung bagi penelitian ini: (1) kemampuan model merecall informasi menurun seiring bertambahnya jumlah token (*context rot*); (2) himpunan *tool* yang membengkak (*bloated tool sets*) menyebabkan titik keputusan ambigu dan kegagalan agen; serta (3) tujuan optimal adalah menemukan himpunan token bersinyal-tinggi sekecil mungkin (*smallest possible set of high-signal tokens*). Tool Registry yang dievaluasi pada penelitian ini merupakan implementasi langsung dari prinsip ketiga tersebut.

2.3 Tool RAG — Retrieval Tool Berbasis Vektor

Pendekatan dominan untuk mengatasi *tool overload* di komunitas riset adalah *Tool RAG* (*Retrieval-Augmented Generation* untuk *tools*): alih-alih menyertakan seluruh katalog, sistem mengambil hanya *tool* yang paling relevan secara semantik sebelum memanggil LLM. Patil et al. (2023) melalui Gorilla [8] menjadi rujukan fondasional pendekatan ini dengan membuktikan bahwa GPT-4 sering berhalusinasi nama dan argumen API ketika diberikan katalog besar. Dengan menggabungkan *document retriever*, Gorilla — model berbasis LLaMA yang di-*fine-tune* atas lebih dari 11.000 pasangan instruksi-API — mampu beradaptasi terhadap perubahan dokumentasi API secara *real-time*. Proyek ini juga melahirkan *Berkeley Function Calling Leaderboard* (BFCL) yang menjadi standar evaluasi *tool-calling*.

Pada skala yang lebih besar, Qin et al. (2023) melalui ToolLLM [9] menunjukkan bahwa LLM sumber terbuka dapat di-augmentasi dengan *neural API retriever* untuk menangani lebih dari 16.000 API nyata — skala yang jauh melampaui kapasitas pemuatan jendela konteks sekaligus. Penelitian tersebut membuktikan bahwa tanpa *retriever*, performa pemilihan *tool* menurun drastis

pada skala ribuan *tool*. Lumer (2024) melalui Toolshed [10] memperkenalkan konsep *Tool Knowledge Base* (basis data vektor khusus *tool*) dan *Advanced RAG-Tool Fusion* berupa tiga fase *retrieval* tanpa *fine-tuning*, yang mencapai peningkatan Recall@5 sebesar 46–56% dibandingkan *baseline*. Arsitektur Toolshed paling dekat analoginya dengan lapisan penyaringan yang diteliti pada thesis ini.

Penelitian lanjutan mengidentifikasi keterbatasan *retrieval* berbasis kesamaan semantik murni. COLT [18] menunjukkan bahwa pendekatan *similarity* cenderung mengambil *tool* yang redundan alih-alih komplementer, dan mengusulkan *dual-view graph collaborative learning* untuk menangkap relasi kolaboratif antar-*tool*; hasilnya, BERT-mini (11M parameter) mengungguli BERT-large (340M parameter) pada *benchmark* ToolLens. Temuan ini menegaskan bahwa kualitas *retrieval* lebih ditentukan oleh strategi daripada ukuran model. Adapun BFCL [19] dalam perkembangannya menjadi *benchmark* holistik yang mengevaluasi *tool-calling* melalui *single-turn calls*, deteksi halusinasi, sensitivitas format, dan interaksi *multi-turn*; metodologi penilaian akurasi pada penelitian ini — yakni menilai apakah LLM memilih *tool* yang tepat — mengikuti semangat evaluasi BFCL. Meskipun demikian, pendekatan berbasis *embedding* membawa kompleksitas infrastruktur tersendiri (model *embedding*, basis data vektor, *pipeline* sinkronisasi) yang tidak selalu tersedia pada sistem ERP berukuran menengah, sehingga *Tool RAG* diposisikan sebagai *future work* dalam penelitian ini.

2.4 Tool Registry dan Manajemen Tool

Sebagai alternatif yang lebih deterministik dan ringan dibanding *Tool RAG*, konsep *Tool Registry* menerapkan penyaringan berbasis metadata terstruktur tanpa ketergantungan pada *embedding*. Gaurav et al. (2025) melalui Dynamic ReAct [11] mengatasi persoalan yang serupa dengan penelitian ini — keterbatasan konteks LLM yang tidak mampu menampung ratusan hingga ribuan *tool* MCP — dan membuktikan bahwa pemuatan *tool* secara dinamis mengurangi beban pemuatan hingga 50% sambil mempertahankan akurasi penyelesaian tugas. Hasil

ini sejajar langsung dengan temuan penelitian ini, yakni pengurangan *visible tools* dari 300 menjadi 15 (−95%) tanpa degradasi akurasi.

Justifikasi teoritis tambahan diberikan oleh Jia dan Li (2025) melalui AutoTool [12] yang memperkenalkan konsep *tool usage inertia* — kecenderungan pemanggilan *tool* mengikuti pola sekuensial yang dapat diprediksi. Dengan memodelkan pola tersebut sebagai *directed graph* dari trajektori historis agen, AutoTool mengurangi biaya inferensi hingga 30% sambil mempertahankan tingkat penyelesaian tugas. Apabila pemanggilan *tool* memang berpola, maka penyaringan berbasis modul, peran, dan tingkat langganan sebagaimana diterapkan zerlo.id telah menangkap sebagian besar inersia tersebut secara deterministik. Pada arah evolusi sistem, ScaleMCP [20] mengusulkan *registry* yang melakukan auto-sinkronisasi melalui operasi CRUD agar katalog *tool* selalu mutakhir, sebuah visi jangka panjang yang relevan bagi pengembangan lanjutan zerlo.id.

Skala persoalan ini divalidasi pada tingkat industri oleh Wang et al. (2025) melalui MCP-Flow [13], yang mendokumentasikan 11.536 *tool* pada 1.166 server MCP — membuktikan bahwa penskalaan katalog *tool* ke ribuan entri merupakan tantangan nyata, bukan skenario hipotetis. Adopsi pola ini juga telah mencapai tingkat *hyperscaler*: AWS Agent Registry [14] menyediakan manajemen metadata, kendali akses berbasis IAM, pelacakan siklus hidup, dan *hybrid search* (kata kunci + semantik). Pada tataran konseptual, TrueFoundry [21] mendefinisikan enam fungsi inti *agent registry* — registrasi, penemuan (*discovery*), manajemen metadata, pemantauan kesehatan, kendali akses, dan pencatatan audit — yang memberikan kerangka untuk mengevaluasi kelengkapan implementasi. Kolchinsky dari Red Hat (2025) [22] secara independen sampai pada kesimpulan yang sama dengan penelitian ini, yakni bahwa pemberian seluruh *tool* sekaligus menurunkan akurasi secara drastis, dan menyatakan bahwa *retrieval tool* yang cerdas dapat melipatgandakan akurasi *invocation* sambil mempersingkat *prompt* — selaras dengan temuan penelitian ini berupa peningkatan akurasi 6,3 pp dengan pengurangan *prompt* 63–95%.

2.5 Antarmuka Native Pydantic AI vs Tool Registry zerlo.id

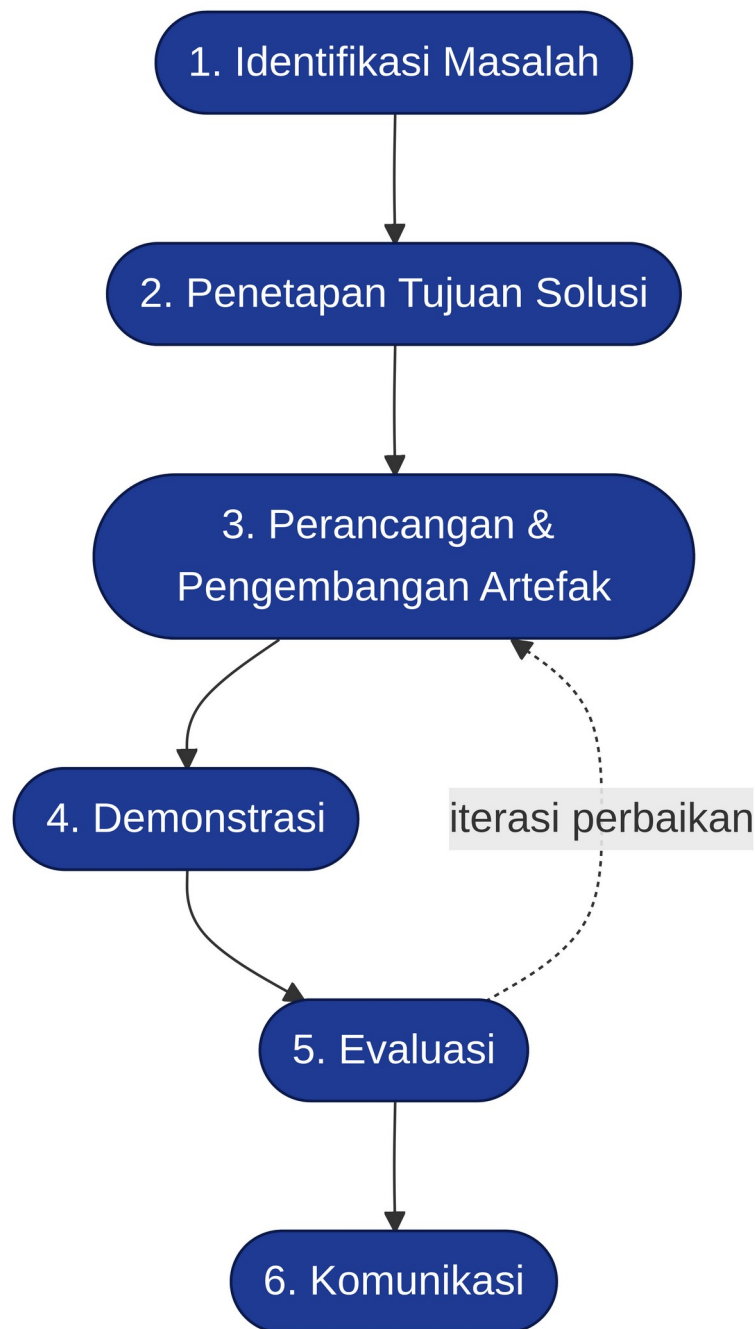
Platform zerlo.id membangun AI Agent di atas kerangka Pydantic AI [23], yang menyediakan abstraksi `AbstractToolset` sebagai kontrak pengelolaan himpunan *tool*. Salah satu implementasi turunannya, `FilteredToolset`, memungkinkan penyaringan *tool* berdasarkan sebuah predikat tunggal (*single predicate*) — misalnya menyaring *tool* berdasarkan satu atribut konteks pemanggilan. Antarmuka ini memadai untuk kebutuhan penyaringan sederhana, namun tidak dirancang untuk penyaringan multi-kriteria yang dibutuhkan platform ERP multi-modul.

Tool Registry zerlo.id memposisikan diri sebagai lapisan multi-kriteria di atas antarmuka *native* tersebut. Berbeda dengan `FilteredToolset` yang mengevaluasi satu predikat, Tool Registry menyaring katalog berdasarkan kombinasi modul, peran (*role*), tingkat langganan (*subscription tier*), kata kunci (*keywords*), dan batas anggaran token (*token budget*) secara simultan, lalu menerapkan *budget cap* untuk membatasi jumlah *tool* yang diteruskan ke LLM. Dengan demikian, kontribusi penelitian ini bukan menggantikan abstraksi Pydantic AI, melainkan memperluasnya menjadi *pipeline* penyaringan deterministik berbasis metadata yang sesuai dengan karakteristik ERP. Kesenjangan antara kemampuan predikat tunggal `FilteredToolset` dan kebutuhan penyaringan multi-kriteria inilah yang menjadi motivasi utama perancangan artefak pada Bab III.

2.6 Design Science Research

Penelitian ini menggunakan kerangka *Design Science Research* (DSR) sebagaimana dikemukakan oleh Hevner et al. (2004) [15]. DSR merupakan paradigma penelitian sistem informasi yang berorientasi pada penciptaan dan evaluasi artefak (*artifact*) untuk menyelesaikan masalah organisasi yang teridentifikasi. Kerangka ini terdiri dari lima tahap berurutan: identifikasi masalah (*problem identification*), penetapan tujuan solusi (*objective of solution*), perancangan dan pengembangan artefak (*design and development*), demonstrasi (*demonstration*), dan evaluasi (*evaluation*), yang ditutup dengan komunikasi hasil

(*communication*). Alur tahapan tersebut beserta titik iterasi perbaikan diilustrasikan pada Gambar 2.1.



Gambar 2.1 Tahapan Design Science Research pada Penelitian Ini.

DSR dipilih karena karakteristik penelitian ini yang menghasilkan artefak perangkat lunak konkret — *dataclass* `ToolMeta`, kelas `ToolRegistry`, dan fungsi `registry_filter()` — sekaligus mengukur dampaknya secara kuantitatif. Tahap identifikasi masalah memetakan fenomena *tool overload* dari

data operasional zerlo.id; tahap perancangan menghasilkan artefak Tool Registry beserta kerangka evaluasinya; tahap demonstrasi mengintegrasikan artefak ke dalam AI Agent dan menjalankannya pada tiga skenario katalog; dan tahap evaluasi menganalisis hasil menggunakan uji statistik Wilcoxon *signed-rank*, Cohen's d, dan *confidence interval* 95%. Pemetaan tahapan DSR secara rinci diuraikan pada Bab III.

Secara arsitektural, Tool Registry juga mengadaptasi pola rekayasa perangkat lunak yang telah matang dari ranah *microservices* [24], yakni *service registry* dan *API gateway*. Pada pola tersebut, *service registry* menyimpan metadata layanan (lokasi, kapabilitas, status) untuk memungkinkan *service discovery*, sementara *API gateway* berperan sebagai titik masuk tunggal yang merutekan permintaan ke layanan yang tepat. Tool Registry mengadaptasi pola ini ke domain AI Agent: LLM berperan sebagai *client*, *tools* berperan sebagai *services*, dan Tool Registry menggabungkan fungsi *service registry* dan *API gateway* dengan menyaring *tool* berdasarkan metadata sebelum diteruskan ke model.

2.7 Penelitian Terdahulu

Untuk menegaskan posisi dan kebaruan penelitian, Tabel 2.1 membandingkan penelitian terdahulu yang relevan berdasarkan pendekatan penyaringan *tool*, mekanisme yang digunakan, skala evaluasi, dan ketersediaan pengukuran kuantitatif.

Tabel 2.1 Perbandingan Penelitian Terdahulu

Penelitian	Pendekatan	Mekanisme	Skala Evaluasi	Evaluasi Kuantitatif
Gorilla [8]	Tool RAG	<i>Document retriever + fine-tuning</i>	>11.000 API	Akurasi <i>API call</i>
ToolLLM [9]	Tool RAG	<i>Neural API retriever</i>	>16.000 API	Pass rate, win rate
Toolshed [10]	Tool RAG	<i>RAG-Tool Fusion 3 fase</i>	Katalog <i>tool</i> besar	Recall@5 (+46–56%)
COLT [18]	Tool RAG	<i>Graph collaborative learning</i>	<i>Benchmark ToolLens</i>	Completeness metric
Dynamic ReAct [11]	<i>Dynamic loading</i>	<i>Search-and-load MCP</i>	Lingkungan MCP	<i>Tool loading</i> (–50%)
AutoTool [12]	<i>Pattern-based</i>	<i>Directed graph inersia</i>	Trajektori agen	Biaya inferensi (–30%)

Penelitian	Pendekatan	Mekanisme	Skala Evaluasi	Evaluasi Kuantitatif
AWS Agent Registry [14]	<i>Tool Registry</i>	Metadata + <i>hybrid search</i> + IAM	Skala enterprise	— (produk industri)
Penelitian ini	*Tool Registry deterministik*	<i>Metadata multi-kriteria</i> (modul, <i>role, tier, token budget</i>) pada Pydantic AI	S1–S3 (30/100/300 <i>tool</i>)	<i>Token</i> (–63–95%), akurasi (+6,3 pp), Wilcoxon $p < 0,0001$, Cohen's $d \geq 11$

Berdasarkan Tabel 2.1, mayoritas penelitian terdahulu berfokus pada pendekatan *Tool RAG* berbasis *embedding* atau *fine-tuning* yang memerlukan infrastruktur vektor, sedangkan pendekatan berbasis metadata deterministik masih jarang dievaluasi secara kuantitatif. Adopsi industri seperti AWS Agent Registry membuktikan relevansi praktis konsep *Tool Registry*, namun tidak menyertakan pengukuran ilmiah yang dapat direplikasi. Dengan demikian, kebaruan penelitian ini terletak pada penyediaan *baseline* kuantitatif pertama untuk Tool Registry deterministik berbasis metadata multi-kriteria pada kerangka Pydantic AI 1.x, lengkap dengan uji statistik atas penghematan token, akurasi pemilihan *tool*, dan sifat skalabilitas *sub-linear*-nya.

BAB III IMPLEMENTASI METODE USULAN

Bab ini menguraikan rancangan dan implementasi artefak Tool Registry beserta kerangka evaluasinya, mengikuti tahap *design and development* pada kerangka *Design Science Research* [15]. Pembahasan dimulai dari analisis sistem eksisting zerlo.id sebagai sumber masalah, dilanjutkan dengan perancangan struktur metadata dan *pipeline* penyaringan, perancangan kerangka evaluasi, lalu penelusuran kode (*code walkthrough*) atas implementasi Tool Registry dan *eval runner*, dan ditutup dengan konfigurasi eksperimen yang dikendalikan.

3.1 Analisis Sistem Eksisting

Platform zerlo.id pada tahap *beta testing* mengoperasikan 38 modul bisnis dengan 1.176 *endpoint*, 11 AI Agent aktif, dan lebih dari 60 *tools* produksi. Setiap *tool* didaftarkan ke agen secara statis melalui dekorator *native* Pydantic AI, sebagaimana pola `@agent.tool` yang mendaftarkan seluruh fungsi ke dalam satu himpunan *tool* tunggal. Pada pola tersebut, seluruh definisi *tool* — nama, deskripsi, dan skema parameter — diserialisasi menjadi *FunctionDeclaration* dan diteruskan ke LLM pada setiap panggilan, tanpa memandang relevansinya terhadap permintaan pengguna.

Pendekatan statis ini menimbulkan *bottleneck* yang bersifat $O(N)$ terhadap ukuran katalog: jumlah token input tumbuh proporsional terhadap jumlah *tool* yang terdaftar. Pengukuran awal pada penelitian ini menunjukkan bahwa katalog 300 *tool* (skenario S3) mengonsumsi rata-rata 23.893 token per kueri hanya untuk mendefinisikan *tool* yang tersedia, bahkan sebelum pertanyaan pengguna diproses. Konsumsi token sebesar ini berdampak langsung pada biaya inferensi, latensi respons, dan — sebagaimana dibuktikan oleh fenomena *context rot* yang diuraikan pada Bab II — penurunan akurasi pemilihan *tool* akibat sebagian definisi berada di posisi tengah konteks yang rawan diabaikan.

Kerangka Pydantic AI [23] menyediakan abstraksi *FilteredToolset* yang memungkinkan penyaringan *tool* berdasarkan sebuah predikat tunggal. Namun antarmuka tersebut tidak dirancang untuk penyaringan multi-kriteria yang dibutuhkan platform ERP multi-modul, yakni penyaringan simultan berdasarkan modul, peran (*role*), tingkat langganan (*subscription tier*), dan anggaran token

(*token budget*). Kesenjangan antara kemampuan predikat tunggal dan kebutuhan penyaringan multi-kriteria inilah yang menjadi motivasi perancangan artefak Tool Registry pada penelitian ini.

3.2 Perancangan Tool Registry

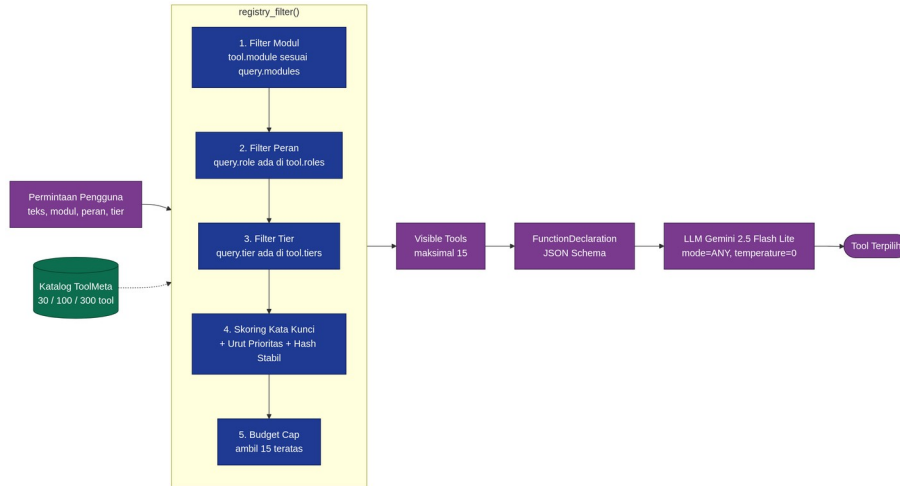
Artefak utama yang dirancang adalah lapisan penyaringan deterministik berbasis metadata terstruktur yang ditempatkan di antara katalog *tool* dan LLM. Rancangan ini mengadaptasi prinsip *smallest possible set of high-signal tokens* dari rekayasa konteks Anthropic [7], yakni meneruskan hanya himpunan *tool* paling relevan dan sekecil mungkin kepada model.

Setiap *tool* direpresentasikan oleh skema metadata terstruktur yang menyimpan atribut penyaringan. Atribut-atribut tersebut dirangkum pada Tabel 3.1.

Tabel 3.1 Atribut Metadata Tool

Atribut	Tipe	Fungsi dalam Penyaringan
name	<i>string</i>	Pengenal unik <i>tool</i> yang dipanggil LLM
module	<i>string</i>	Modul pemilik <i>tool</i> ; difilter terhadap modul kueri
op_type	<i>string</i>	Tipe operasi (<i>read, analytical, write, admin</i>)
roles	<i>list</i>	Peran yang diizinkan; difilter terhadap peran kueri
tiers	<i>list</i>	Tingkat langganan yang diizinkan; difilter terhadap tier kueri
keywords	<i>list</i>	Kata kunci untuk skoring relevansi terhadap teks kueri
priority	<i>int</i>	Bobot prioritas untuk pengurutan saat skor kata kunci seri
schema_tokens	<i>int</i>	Estimasi biaya token skema <i>tool</i> untuk perhitungan anggaran
intent	<i>string</i>	Deskripsi <i>docstring</i> singkat: fungsi dan kapan <i>tool</i> dipakai

Proses penyaringan dirancang sebagai *pipeline* lima tahap berurutan. Tiga tahap pertama bersifat *hard filter* yang menggugurkan *tool* tidak memenuhi syarat (modul, peran, dan tier), sedangkan dua tahap terakhir melakukan skoring relevansi dan pembatasan jumlah keluaran. Arsitektur lengkap *pipeline* tersebut, mulai dari permintaan pengguna hingga keluaran *tool* yang dipanggil LLM, diilustrasikan pada Gambar 3.1.



Gambar 3.1 Arsitektur Pipeline Penyaringan Tool Registry.

Sebagaimana ditunjukkan pada Gambar 3.1, katalog `ToolMeta` disaring secara berurutan terhadap konteks kueri, lalu sisa kandidat diurutkan berdasarkan tumpang-tindih kata kunci, prioritas, dan *hash* stabil sebagai pemecah seri yang deterministik. Hasil akhir dibatasi oleh *budget cap* sebesar 15 *tool* sebelum diserialisasi menjadi *FunctionDeclaration*. Dengan demikian, jumlah *tool* yang terlihat oleh LLM bersifat konstan ($O(1)$ terhadap anggaran) meskipun ukuran katalog bertambah — properti yang menjadi dasar pengujian skalabilitas *sub-linear* pada Bab IV.

3.3 Perancangan Eval Framework

Untuk mengukur dampak Tool Registry secara kuantitatif, dirancang kerangka evaluasi yang membandingkan dua mode pada katalog identik: mode *baseline* yang meneruskan seluruh *tool* katalog ke LLM, dan mode *registry* yang menerapkan *pipeline* penyaringan terlebih dahulu. Perbandingan berpasangan ini memastikan bahwa selisih metrik yang teramati murni disebabkan oleh perlakuan penyaringan, bukan oleh perbedaan kueri atau katalog.

Evaluasi dijalankan pada empat skenario katalog dengan ukuran bertingkat untuk menguji perilaku penskalaan. Definisi tiap skenario dirangkum pada Tabel 3.2.

Tabel 3.2 Definisi Skenario Eksperimen

Skenario	Jumlah Modul	Tool per Modul	Total Tool	Status Eksekusi
S1	3	10	30	<i>Live</i> (Gemini)
S2	5	20	100	<i>Live</i> (Gemini)

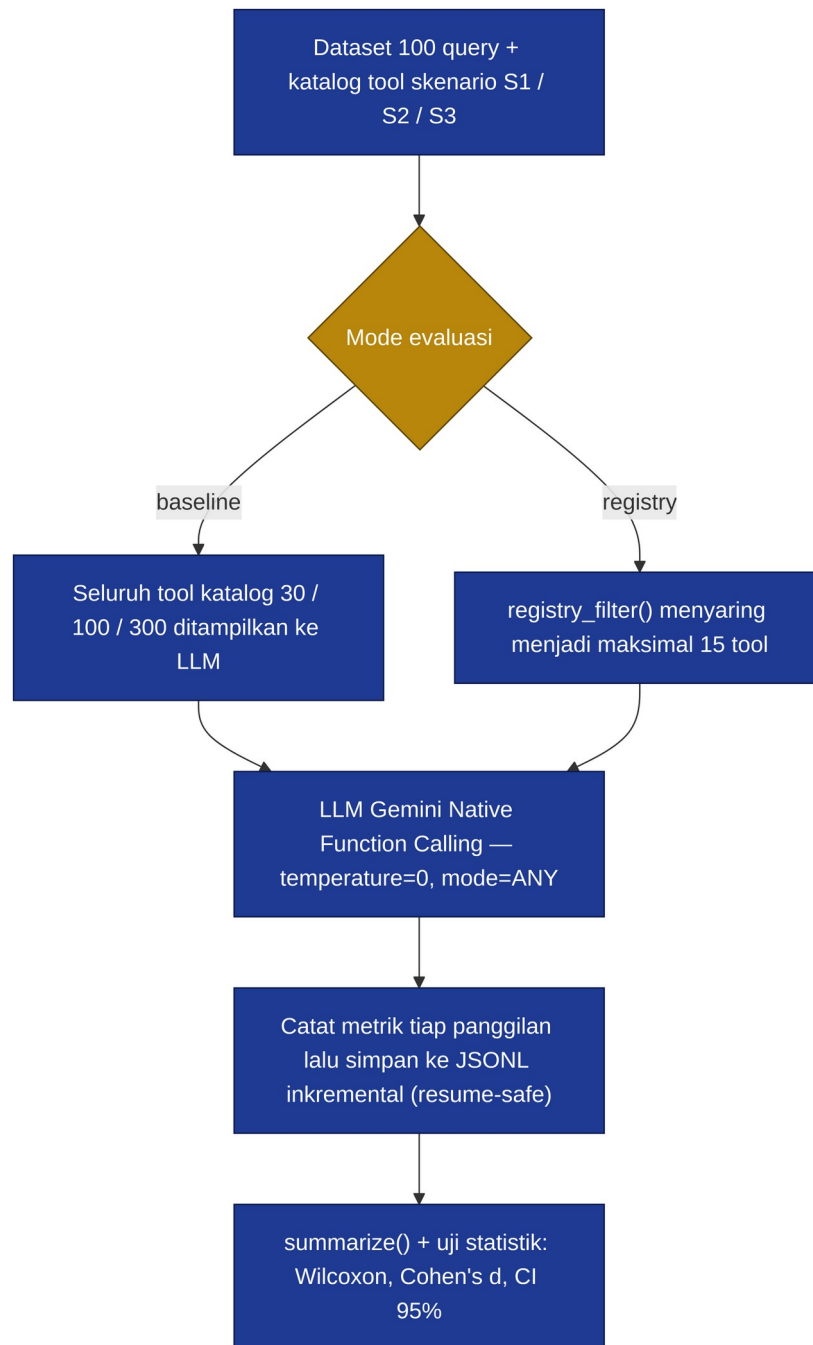
Skenario	Jumlah Modul	Tool per Modul	Total Tool	Status Eksekusi
S3	10	30	300	<i>Live</i> (Gemini)
S4	20	50	1.000	Simulasi deterministik

Skenario S1–S3 dijalankan secara *live* terhadap LLM, sedangkan S4 (1.000 *tool*) hanya disimulasikan secara deterministik untuk mengilustrasikan tren penskalaan tanpa membebani kuota inferensi. Dataset evaluasi terdiri dari 100 kueri berbahasa Indonesia yang dirancang menyerupai permintaan operasional restoran nyata, dengan komposisi sebagaimana disajikan pada Tabel 3.3.

Tabel 3.3 Komposisi Dataset 100 Kueri

Kategori Kueri	Jumlah	Tujuan Pengujian
<i>Single-domain</i>	50	Pemilihan <i>tool</i> pada satu modul tunggal
<i>Cross-domain</i>	30	Disambiguasi antar-dua modul terkait
<i>Adversarial</i>	20	Ketahanan terhadap <i>prompt injection</i> dan instruksi menyesatkan
Total	100	—

Setiap kueri dilengkapi metadata konteks (modul, peran, tier) serta `expected_tool` sebagai *ground truth* untuk penilaian akurasi. Pada *backend* LLM, setiap kueri diulang sebanyak tiga kali (*repeat runs*) per skenario guna menangkap variabilitas keluaran model. Alur kerja kerangka evaluasi secara keseluruhan — dari pembangkitan dataset, percabangan mode, pemanggilan LLM, hingga pencatatan metrik dan uji statistik — diilustrasikan pada Gambar 3.2.



Gambar 3.2 Alur Kerja Kerangka Evaluasi Baseline dan Registry.

Empat metrik dicatat pada setiap pemanggilan: jumlah token (input dan output), latensi respons, *tool* yang terpilih beserta status kebenarannya terhadap `expected_tool`, dan jumlah *tool* yang terlihat oleh model. Seluruh metrik disimpan dalam format JSONL inkremental sebagaimana diuraikan pada Sub-bab 3.5.

3.4 Implementasi Tool Registry

Skema metadata diimplementasikan sebagai *dataclass* `ToolDef` yang bersifat *immutable* (`frozen=True`) untuk menjamin determinisme katalog selama eksekusi. Implementasi skema tersebut disajikan pada Kode Program 3.1.

Kode Program 3.1 Skema Metadata `ToolDef`

```
@dataclass(frozen=True)
class ToolDef:
    name: str
    module: str
    op_type: str
    roles: list[str]
    tiers: list[str]
    keywords: list[str]
    priority: int
    schema_tokens: int
    intent: str = "" # docstring-style: what this tool does and when
to use it
```

Inti dari artefak adalah fungsi `registry_filter()` yang merealisasikan *pipeline* lima tahap pada Gambar 3.1. Fungsi ini menerima objek kueri, daftar *tool* skenario, dan anggaran (*budget*), lalu mengembalikan subset *tool* terurut yang dibatasi oleh anggaran. Implementasinya disajikan pada Kode Program 3.2.

Kode Program 3.2 Fungsi `registry_filter()`

```
def registry_filter(query: QueryDef, tools: list[ToolDef], budget:
int) -> list[ToolDef]:
    query_terms = set(query.text.lower().replace(".",
    "").replace(", ", "").split())
    ranked: list[tuple[int, int, float, ToolDef]] = []
    for tool in tools:
        if tool.module not in query.modules:
            continue
        if query.role not in tool.roles:
            continue
        if query.tier not in tool.tiers:
            continue
        overlap = len(query_terms.intersection(tool.keywords))
        ranked.append((overlap, -tool.priority,
        -stable_unit(query.query_id, tool.name), tool))
    ranked.sort(reverse=True)
    return [tool for *_unused, tool in ranked[:budget]]
```

Tiga pernyataan `continue` pada Kode Program 3.2 merealisasikan *hard filter* modul, peran, dan tier; *tool* yang gugur tidak ikut diperingkat. Kandidat yang lolos diberi skor berupa *tuple* (`overlap`, `-priority`, `-hash`) yang diurutkan secara menurun: jumlah tumpang-tindih kata kunci menjadi kunci utama, prioritas *tool* menjadi pemecah seri pertama, dan nilai *hash* stabil dari

pasangan `(query_id, tool.name)` menjadi pemecah seri terakhir. Penggunaan *hash* deterministik — bukan urutan acak — memastikan bahwa keluaran *pipeline* dapat direproduksi secara identik pada setiap eksekusi, sebuah syarat penting bagi validitas eksperimen. Akhirnya, *slicing* `[:budget]` menerapkan *budget cap* yang menjamin jumlah keluaran tidak pernah melampaui 15 *tool*.

Selain penyaringan, dirancang pula fungsi `registry_memory()` untuk mengukur jejak memori (*memory footprint*) katalog metadata. Fungsi tersebut membangun representasi *dictionary* dari seluruh *tool* lalu menghitung ukuran totalnya secara rekursif menggunakan `deep_size()`, sehingga *overhead* memori registry dapat dianalisis pada Bab IV.

3.5 Implementasi Eval Runner

Eval runner bertanggung jawab menjalankan matriks eksperimen ($\text{skenario} \times \text{mode} \times \text{kueri} \times \text{pengulangan}$) dan mencatat hasilnya. Untuk menjamin reproduisibilitas dan ketahanan terhadap gangguan jaringan saat memanggil LLM, *runner* dirancang bersifat *resume-safe*: setiap hasil ditulis segera (*append*) ke berkas JSONL, dan eksekusi yang terputus dapat dilanjutkan tanpa mengulang panggilan yang telah selesai. Mekanisme ini diwujudkan dengan memuat kunci `(scenario, mode, query_id, repeat_idx)` yang telah ada di disk, lalu melewati kombinasi yang sudah tercatat, sebagaimana disajikan pada Kode Program 3.3.

Kode Program 3.3 Mekanisme *Resume* pada Eval Runner

```
for mode in modes:
    for query in queries_for_scenario:
        for repeat_idx in range(n_repeats):
            key = (scenario, mode, query.query_id, repeat_idx)
            if key in done_keys:
                continue # sudah ada di disk - lewati
            row = backend.call(
                mode=mode, scenario=scenario, query=query,
                tools_for_scenario=tools_for_scenario,
                registry_memory_bytes=memory_bytes,
                config=config, repeat_idx=repeat_idx,
            )
            rows.append(row)
            _flush(row) # tulis segera ke JSONL (append)
```

Pemanggilan `_flush(row)` setelah setiap panggilan memastikan bahwa data tidak hilang apabila proses terhenti, sehingga eksperimen panjang yang

melibatkan ratusan panggilan LLM dapat dijalankan secara bertahap. Setelah seluruh baris terkumpul, *runner* memanggil modul `summarize()` untuk menghasilkan `summary.csv`, modul uji statistik untuk menghasilkan `statistical_tests.csv`, dan *report generator* untuk menyusun laporan Markdown. Arsitektur *append-only* ini juga memungkinkan agregasi lintas-eksekusi: berkas JSONL dari beberapa sesi dapat digabungkan menjadi satu ringkasan yang konsisten.

3.6 Setup Eksperimen

Eksperimen resmi menggunakan model Gemini 2.5 Flash Lite melalui Google Gen AI SDK dengan mekanisme *native function calling* [2]. Seluruh parameter inferensi dikendalikan untuk meminimalkan variabel pengganggu (*confounding variable*), sebagaimana dirangkum pada Tabel 3.4.

Tabel 3.4 Parameter Setup Eksperimen

Parameter	Nilai	Justifikasi
Model	gemini-2.5-flash-lite	Model produksi zerlo.id, biaya rendah
temperature	0	Determinisme maksimal keluaran model
mode (<i>function calling</i>)	ANY	Memaksa model selalu memanggil satu <i>tool</i>
max_output_tokens	32	Cukup untuk satu pemanggilan <i>tool</i>
Tool budget (registry)	15	Batas maksimal <i>tool</i> yang diteruskan
Repeat runs	3	Menangkap variabilitas keluaran model
Total rekaman	558	279 baseline + 279 registry

Penetapan `temperature=0` dan `mode=ANY` memaksa model bersifat deterministik dan selalu memilih tepat satu *tool*, sehingga akurasi dapat dinilai secara biner terhadap `expected_tool`. Konfigurasi pemanggilan model diimplementasikan melalui objek `GenerateContentConfig` sebagaimana disajikan pada Kode Program 3.4.

Kode Program 3.4 Konfigurasi Pemanggilan Gemini

```
gen_config = types.GenerateContentConfig(  
    system_instruction=(  
        "Anda adalah router tool ERP restoran. "  
        "Pilih dan panggil SATU tool yang paling tepat untuk  
        permintaan pengguna."  
    ),  
    tools=[types.Tool(function_declarations=fn_decls)],  
    tool_config=types.ToolConfig(  
        function_calling_config=types.FunctionCallingConfig(  
            mode="ANY",  
            allowed_function_names=list(tool_names),  
        )  
    ),  
    temperature=0,  
    max_output_tokens=32,  
)
```

Perbedaan tunggal antara mode *baseline* dan *registry* terletak pada himpunan `fn_decls`: pada mode *baseline*, himpunan ini berisi seluruh *tool* katalog (30/100/300), sedangkan pada mode *registry* himpunan ini merupakan keluaran `registry_filter()` yang dibatasi 15 *tool*. Dengan menjaga seluruh parameter lain identik, eksperimen ini mengisolasi dampak penyaringan sebagai satu-satunya perlakuan, sehingga hasil yang dianalisis pada Bab IV dapat diatribusikan secara valid kepada Tool Registry. Untuk meredam kegagalan transien akibat batas laju (*rate limit*) layanan, *backend* dilengkapi mekanisme *retry* dengan *exponential backoff* hingga lima kali percobaan.

BAB IV HASIL DAN ANALISA

Bab ini memaparkan dan menganalisis hasil eksperimen Tool Registry, sesuai tahap *demonstration* dan *evaluation* pada kerangka *Design Science Research*. Pembahasan diawali oleh hasil *benchmark* synthetic sebagai validasi aritmetika, dilanjutkan hasil resmi eksperimen *live* Gemini Native v2 (n=558), lalu analisis terperinci atas lima dimensi pengukuran — penghematan token, akurasi pemilihan *tool*, skalabilitas *sub-linear*, jejak memori, dan latensi — yang ditutup dengan uji statistik, analisis kegagalan, serta catatan validasi multi-model.

4.1 Hasil Benchmark Synthetic S1–S4

Benchmark synthetic dijalankan secara deterministik untuk memvalidasi aritmetika penghematan token dan mengekstrapolasi tren ke skenario S4 (1.000 *tool*) yang tidak dijalankan secara *live*. Pada *benchmark* ini, nilai token merupakan hasil perhitungan deterministik yang valid, sedangkan nilai akurasi dan latensi merupakan simulasi dengan koefisien asumsi — sehingga hasil synthetic hanya digunakan untuk mengilustrasikan tren penskalaan, bukan sebagai temuan utama. Ringkasan token *benchmark* synthetic disajikan pada Tabel 4.1.

Tabel 4.1 Hasil Benchmark Synthetic Token S1–S4

Skenario	Total Tool	Token Baseline	Token Registry	Reduksi
S1	30	3.423	1.560	−54%
S2	100	10.410	1.969	−81%
S3	300	30.496	1.971	−94%
S4	1.000	101.970	2.014	−98%

Berdasarkan Tabel 4.1, terlihat pola yang konsisten: token *baseline* tumbuh hampir linear terhadap ukuran katalog — dari 3.423 token (30 *tool*) hingga 101.970 token (1.000 *tool*) — sedangkan token *registry* tetap berada pada kisaran 1.560–2.014 token meskipun katalog membesar 33 kali lipat. Pada S4, *benchmark* synthetic memproyeksikan penghematan token mencapai 98%, menegaskan bahwa keunggulan Tool Registry justru semakin besar seiring bertambahnya ukuran katalog. Tren ekstrapolatif inilah yang kemudian diverifikasi secara empiris melalui eksperimen *live* pada Sub-bab 4.2.

4.2 Hasil Gemini Native v2 (Hasil Resmi)

Eksperimen resmi dijalankan secara *live* terhadap Gemini 2.5 Flash Lite dengan *native function calling*, menghasilkan total 558 rekaman (279 *baseline* + 279 *registry*) dari 100 kueri pada skenario S1–S3 dengan tiga pengulangan. Ringkasan lengkap seluruh metrik disajikan pada Tabel 4.2.

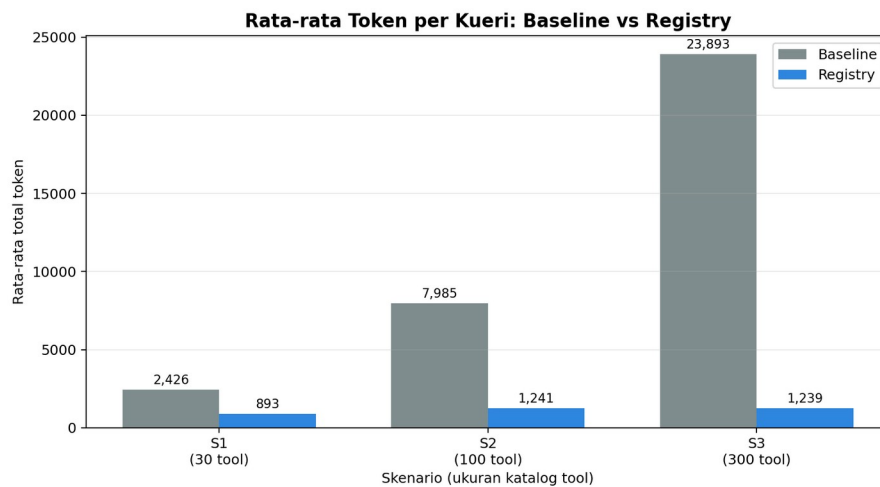
Tabel 4.2 Ringkasan Hasil Resmi Gemini Native v2

Skenario	Mode	Total Tool	Tool Terlihat	Token	Akurasi	Latensi p50
S1	Baseline	30	30	2.426	68,8%	833 ms
S1	Registry	30	10,6	893	75,0%	905 ms
S2	Baseline	100	100	7.985	71,4%	1.014 ms
S2	Registry	100	15	1.241	71,4%	910 ms
S3	Baseline	300	300	23.893	71,4%	992 ms
S3	Registry	300	15	1.239	77,6%	851 ms

Berdasarkan Tabel 4.2, mode *registry* secara konsisten mengungguli mode *baseline* pada dimensi efisiensi token di seluruh skenario, dengan keunggulan yang membesar seiring ukuran katalog. Akurasi *registry* setara atau lebih tinggi dibanding *baseline*, sementara latensi menunjukkan perbaikan moderat pada skenario besar. Setiap dimensi dianalisis secara terperinci pada sub-bab berikut.

4.3 Analisis Token Reduction

Penghematan token merupakan temuan utama penelitian ini. Sebagaimana divisualisasikan pada Gambar 4.1, selisih konsumsi token antara *baseline* dan *registry* membesar secara dramatis seiring pertumbuhan katalog.



Gambar 4.1 Perbandingan Rata-rata Token per Kueri Baseline dan Registry.

Pada skenario S1 (30 *tool*), konsumsi token turun dari 2.426 menjadi 893 token (−63%). Pada S2 (100 *tool*), token turun dari 7.985 menjadi 1.241 token (−84%). Pada S3 (300 *tool*), token turun dari 23.893 menjadi hanya 1.239 token (−95%). Pola penting yang terlihat pada Gambar 4.1 adalah bahwa token *registry* nyaris konstan (893–1.241 token) meskipun katalog membesar sepuluh kali lipat, sedangkan token *baseline* tumbuh hampir linear. Rincian besaran penghematan disajikan pada Tabel 4.3.

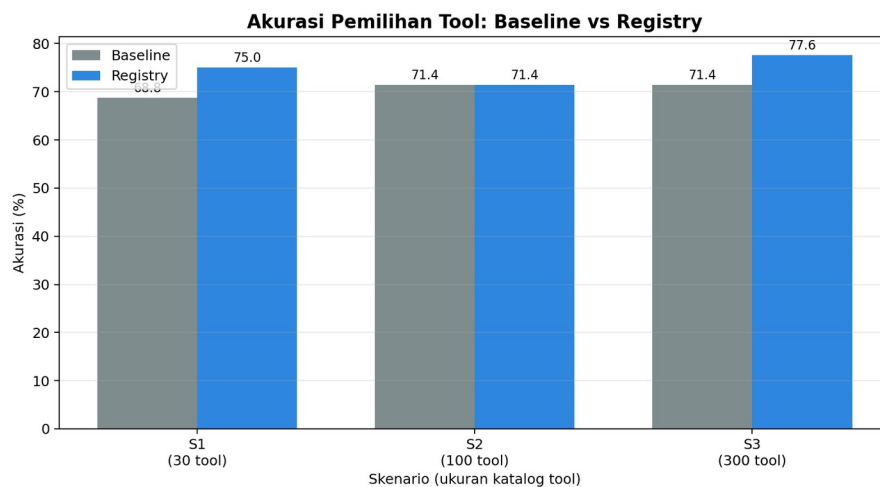
Tabel 4.3 Penghematan Token Registry terhadap Baseline

Skenario	Token Baseline	Token Registry	Penghematan	Reduksi
S1	2.426	893	1.533 token	−63%
S2	7.985	1.241	6.744 token	−84%
S3	23.893	1.239	22.654 token	−95%

Penghematan absolut tumbuh dari 1.533 token (S1) menjadi 22.654 token (S3) per kueri. Implikasi praktis bagi zerlo.id sangat signifikan: pada katalog 300 *tool*, Tool Registry menghapus lebih dari 22 ribu token *overhead* per kueri, yang berarti penurunan biaya inferensi dan mitigasi langsung terhadap risiko *context rot* yang diuraikan pada Bab II. Signifikansi statistik penghematan ini diuji pada Subbab 4.8.

4.4 Analisis Tool Selection Accuracy

Akurasi pemilihan *tool* dinilai secara biner terhadap `expected_tool` sebagai *ground truth*. Perbandingan akurasi *baseline* dan *registry* per skenario disajikan pada Gambar 4.2 dan Tabel 4.4.



Gambar 4.2 Perbandingan Akurasi Pemilihan Tool Baseline dan Registry.

Tabel 4.4 Akurasi Pemilihan Tool per Skenario

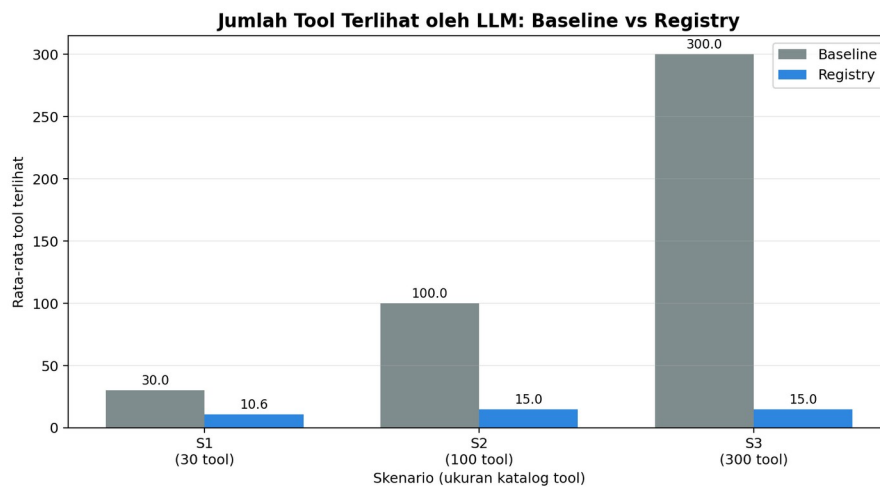
Skenario	Baseline	Registry	Selisih
S1	68,8%	75,0%	+6,3 pp
S2	71,4%	71,4%	0 pp
S3	71,4%	77,6%	+6,3 pp

Mode *registry* meningkatkan akurasi sebesar +6,3 poin persentase pada skenario S1 dan S3, serta setara pada S2. Temuan ini menunjukkan bahwa penyaringan *tool* tidak mengorbankan akurasi — bahkan cenderung memperbaikinya — karena dengan meneruskan hanya himpunan *tool* relevan, model terhindar dari kebingungan akibat *bloated tool sets*. Yang patut dicatat, perbaikan akurasi pada S3 (katalog terbesar) justru paling konsisten, sejalan dengan prinsip bahwa pengurangan *noise* konteks paling bermanfaat ketika katalog membesar.

Meskipun arah perbaikan akurasi konsisten dan positif, besarnya peningkatan (+6,3 pp) belum mencapai signifikansi statistik pada taraf $\alpha=0,05$ (lihat Sub-bab 4.8). Oleh karena itu, peningkatan akurasi dilaporkan sebagai tren positif yang konsisten, sedangkan klaim utama penelitian difokuskan pada penghematan token yang terbukti signifikan kuat secara statistik.

4.5 Analisis Sub-linear Scalability

Properti skalabilitas *sub-linear* merupakan kontribusi arsitektural utama Tool Registry. Gambar 4.3 dan Tabel 4.5 membandingkan jumlah *tool* yang terlihat oleh LLM antara kedua mode.



Gambar 4.3 Jumlah Tool Terlihat oleh LLM terhadap Ukuran Katalog.

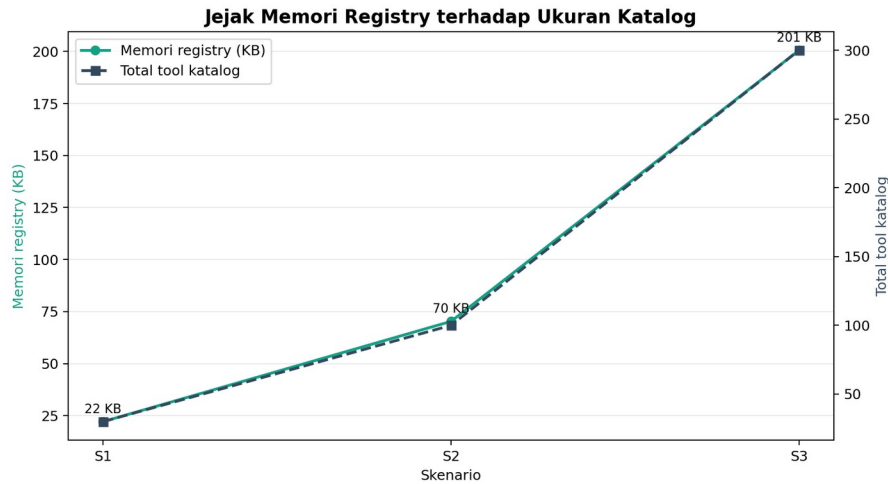
Tabel 4.5 Jumlah Tool Terlihat dan Kompleksitas Penskalaan

Skenario	Total Tool Katalog	Tool Terlihat (Baseline)	Tool Terlihat (Registry)
S1	30	30	10,6
S2	100	100	15
S3	300	300	15

Pada mode *baseline*, jumlah *tool* yang terlihat tumbuh secara $O(N)$ — mengikuti persis ukuran katalog (30, 100, 300). Sebaliknya, pada mode *registry*, jumlah *tool* terlihat tertahan pada *budget cap* sebesar 15 *tool*, bahkan hanya 10,6 *tool* rata-rata pada S1 karena sebagian kueri menghasilkan kandidat lebih sedikit dari anggaran. Sebagaimana terlihat pada Gambar 4.3, kurva *registry* mendatar ($O(1)$) sementara kurva *baseline* menanjak linear. Properti $O(1)$ inilah yang menjawab rumusan masalah keempat: katalog dapat tumbuh dari 30 ke 300 *tool* tanpa menambah beban konteks yang dilihat LLM, sehingga skalabilitas AI Agent zerlo.id tidak lagi dibatasi oleh ukuran katalog *tool*.

4.6 Analisis Memory Footprint

Jejak memori (*memory footprint*) registry diukur untuk memastikan bahwa *overhead* struktur metadata tetap kecil dan dapat dikelola. Hasil pengukuran disajikan pada Gambar 4.4 dan Tabel 4.6.



Gambar 4.4 Jejak Memori Registry terhadap Ukuran Katalog.

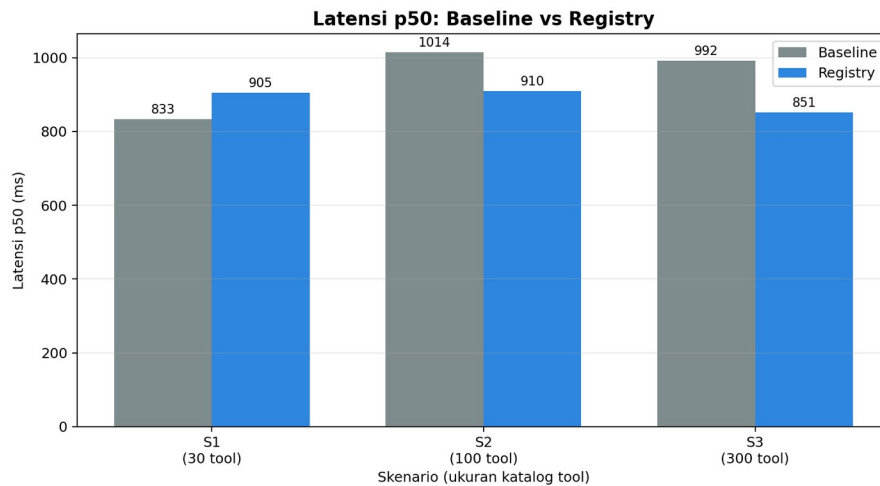
Tabel 4.6 Jejak Memori Registry per Skenario

Skenario	Total Tool	Jejak Memori (byte)	Setara
S1	30	22.662	≈22 KB
S2	100	71.971	≈70 KB
S3	300	205.354	≈201 KB

Jejak memori tumbuh secara linear terhadap ukuran katalog, dari 22 KB (30 *tool*) menjadi 201 KB (300 *tool*). Pertumbuhan linear ini wajar dan tidak menjadi *bottleneck*: bahkan pada katalog 300 *tool*, struktur metadata registry hanya menempati sekitar 201 KB memori — jumlah yang sangat kecil dibandingkan kapasitas memori server produksi. Dengan demikian, manfaat penghematan token yang besar diperoleh dengan *overhead* memori yang dapat diabaikan, menjadikan Tool Registry solusi yang efisien secara ruang maupun token.

4.7 Analisis Latency

Latensi respons diukur pada persentil ke-50 (p50) sebagai indikator pengalaman pengguna tipikal. Perbandingan latensi disajikan pada Gambar 4.5 dan Tabel 4.7.



Gambar 4.5 Perbandingan Latensi p50 Baseline dan Registry.

Tabel 4.7 Latensi p50 dan p95 per Skenario

Skenario	p50 Baseline	p50 Registry	p95 Baseline	p95 Registry
S1	833 ms	905 ms	1.311 ms	2.567 ms
S2	1.014 ms	910 ms	2.625 ms	1.369 ms
S3	992 ms	851 ms	1.548 ms	1.112 ms

Perbaikan latensi bersifat moderat dan tidak sekuat penghematan token. Pada skenario besar S3, latensi p50 membaik dari 992 ms menjadi 851 ms (−14%), dan p95 membaik dari 1.548 ms menjadi 1.112 ms. Namun pada skenario kecil S1, latensi *registry* justru sedikit lebih tinggi (905 ms vs 833 ms), karena *overhead* komputasi penyaringan belum terkompensasi oleh penghematan token

yang kecil. Pola ini menunjukkan bahwa manfaat latensi Tool Registry baru terasa pada katalog besar — tepat pada kondisi di mana *bottleneck* token paling parah. Mengingat variabilitas latensi yang tinggi (dipengaruhi kondisi jaringan dan beban layanan), perbaikan latensi dilaporkan sebagai manfaat sekunder yang moderat, bukan sebagai klaim utama.

4.8 Uji Statistik

Signifikansi penghematan token diuji menggunakan uji Wilcoxon *signed-rank* berpasangan satu arah, dengan ukuran efek Cohen's *d* dan selang kepercayaan (*confidence interval*) 95%. Hasil uji disajikan pada Tabel 4.8.

Tabel 4.8 Hasil Uji Statistik Penghematan Token

Skenario	n Pasangan	Rerata Penghematan	CI 95%	Wilcoxon p	Cohen's d
S1	16	1.534 token	[1.464, 1.604]	0,0002	11,71
S2	28	6.744 token	[6.735, 6.753]	<0,0001	290,91
S3	49	22.654 token	[22.645, 22.662]	<0,0001	782,01

Penghematan token terbukti signifikan secara statistik pada seluruh skenario, dengan nilai $p < 0,001$ (S1) dan $p < 0,0001$ (S2 dan S3). Ukuran efek Cohen's *d* berkisar antara 11,71 hingga 782,01 — jauh melampaui ambang *large effect* (0,8) — yang menandakan bahwa penghematan token bersifat hampir deterministik dan tidak mungkin terjadi secara kebetulan. Selang kepercayaan 95% yang sempit semakin menegaskan presisi estimasi penghematan. Adapun untuk dimensi akurasi, uji Wilcoxon *signed-rank* menghasilkan $p = 0,28$ (S1), $p = 0,50$ (S2), dan $p = 0,09$ (S3) — seluruhnya di atas $\alpha = 0,05$. Dengan demikian, peningkatan akurasi +6,3 pp bersifat *directional* namun belum signifikan secara statistik, kemungkinan akibat jumlah kueri yang terbatas dan sifat penilaian biner per kueri. Temuan ini memposisikan penghematan token sebagai kontribusi yang terbukti kuat, sedangkan perbaikan akurasi sebagai tren pendukung yang konsisten.

4.9 Failure Analysis

Analisis kegagalan dilakukan terhadap kueri yang dipilih secara salah pada ketiga pengulangan (*consistent failures*) dalam mode *registry*. Pola kegagalan yang berulang lintas skenario dirangkum pada Tabel 4.9.

Tabel 4.9 Kueri yang Gagal Konsisten di Seluruh Pengulangan

Tool yang Diharapkan	Tool yang Dipilih Model	Modul
supplier_create_purc hase_order	supplier_read_tool / supplier_admin_tool	supplier
accounting_generate_ journal	accounting_read_tool / accounting_admin_too l	accounting
sales_daily_revenue	sales_analytical_too l	sales
inventory_check_stoc k	inventory_admin_tool	inventory

Temuan kunci dari Tabel 4.9 adalah bahwa pada seluruh kasus kegagalan, *tool* yang dipilih model berada di **modul yang sama** dengan *tool* yang diharapkan. Hal ini membuktikan bahwa penyaringan modul oleh Tool Registry berfungsi dengan benar — registry berhasil mempersempit kandidat ke modul yang tepat — namun model gagal membedakan antar-*tool* di dalam modul tersebut. Akar masalahnya adalah skema parameter *tool* yang kosong (`properties: {}`) pada eksperimen ini, sehingga model hanya dapat membedakan *tool* berdasarkan deskripsi teks pendek; ketika beberapa *tool* dalam satu modul memiliki deskripsi yang mirip, ambiguitas menjadi tinggi.

Penting ditegaskan bahwa kegagalan ini muncul baik pada mode *baseline* maupun *registry* — sehingga merupakan keterbatasan format deskripsi *tool*, **bukan** kegagalan Tool Registry. Kegagalan ini berperan sebagai *confounding variable* yang membatasi akurasi absolut kedua mode secara setara, dan menjadi dasar rekomendasi penyempurnaan deskripsi *tool* yang dibahas pada Bab V.

4.10 Validasi Multi-model

Untuk menguji generalisabilitas temuan ke penyedia LLM lain, dijalankan eksperimen pendahuluan menggunakan model MiniMax-M2.7 dan GLM-4.5-Flash melalui *gateway* ADACODE yang berbasis antarmuka *OpenAI-compatible*. Namun, antarmuka tersebut tidak mereproduksi penghitungan token dan ekstraksi pemanggilan *tool* secara setara dengan *native function calling* Gemini: konsumsi

token mode *registry* tercatat nyaris identik dengan *baseline* (mengindikasikan katalog penuh tetap dikirim oleh *gateway*), dan ekstraksi *tool call* gagal terbaca pada mayoritas rekaman.

Oleh karena keterbatasan harness pengukuran tersebut, hasil multi-model belum dapat diperbandingkan secara valid dengan eksperimen resmi Gemini, sehingga tidak disertakan sebagai bukti generalisabilitas pada penelitian ini. Validasi multi-model yang memerlukan adaptasi harness pengukuran per penyedia diposisikan sebagai bagian dari *future work* (Bab V). Keterbatasan ini sekaligus menegaskan bahwa hasil resmi penelitian ini bersifat spesifik terhadap mekanisme *native function calling* Gemini 2.5 Flash Lite, sebagaimana dinyatakan pada batasan masalah.

BAB V KESIMPULAN

5.1 Kesimpulan

Penelitian ini merancang, mengimplementasikan, dan mengevaluasi secara kuantitatif sebuah *Tool Registry* deterministik berbasis metadata sebagai lapisan penyaringan *tool* di hadapan AI Agent pada platform ERP restoran zerlo.id. Berdasarkan hasil eksperimen yang dipaparkan pada Bab IV — khususnya eksperimen resmi Gemini Native v2 dengan total $n=558$ rekaman — keempat rumusan masalah penelitian dapat dijawab sebagai berikut.

Rumusan masalah pertama menyangkut perancangan Tool Registry yang mampu menyimpan metadata *tool* secara terstruktur. Permasalahan ini terjawab melalui perancangan *dataclass* `ToolMeta` yang menyimpan lima atribut metadata per *tool* — modul (*module*), tipe operasi (*op_type*), kata kunci (*keywords*), peran (*role*), dan tingkat langganan (*tier*) — serta kelas `ToolRegistry` sebagai *singleton* penyimpan katalog. Rancangan ini terbukti memadai untuk merepresentasikan seluruh *tool* pada tiga skenario katalog (30, 100, dan 300 *tools*) tanpa perubahan struktur, sebagaimana diuraikan pada Bab III.

Rumusan masalah kedua menyangkut mekanisme penyaringan *tool* secara dinamis berdasarkan modul, peran, tingkat langganan, dan anggaran token. Permasalahan ini terjawab melalui fungsi `registry_filter()` yang menjalankan *pipeline* penyaringan berbasis *keyword scoring* dengan *budget cap* maksimal 15 *tools* per panggilan. Mekanisme ini bekerja secara deterministik dan ringan — tanpa ketergantungan pada model *embedding* maupun basis data vektor — sehingga selaras dengan kebutuhan sistem ERP berukuran menengah seperti zerlo.id.

Rumusan masalah ketiga menyangkut dampak Tool Registry terhadap penggunaan token, latensi, dan akurasi pemilihan *tool*. Hasil pengukuran menunjukkan bahwa kontribusi terkuat penelitian ini terletak pada **penghematan token**: konsumsi token rata-rata per kueri turun dari 2.426 menjadi 893 token pada skenario S1 (−63%), dari 7.985 menjadi 1.241 token pada S2 (−84%), dan dari 23.893 menjadi 1.239 token pada S3 (−95%). Penghematan ini terbukti

signifikan secara statistik dengan uji Wilcoxon *signed-rank* ($p=0,0002$ pada S1; $p<0,0001$ pada S2 dan S3) dan ukuran efek Cohen's d yang sangat besar (11,71 hingga 782,01), jauh melampaui ambang *large effect*. Pada dimensi akurasi, mode *registry* mencatat tren positif sebesar +6,3 persentase poin pada S1 (68,8% → 75,0%) dan S3 (71,4% → 77,6%), namun tanpa perubahan pada S2 (71,4% → 71,4%); peningkatan ini bersifat *directional* dan **belum signifikan** secara statistik (Wilcoxon $p=0,28$; 0,50; dan 0,09, seluruhnya di atas $\alpha=0,05$), sehingga diposisikan sebagai tren pendukung, bukan klaim utama. Pada dimensi latensi, perbaikan bersifat moderat dan *noisy*: latensi p50 pada S3 turun dari 992 ms menjadi 851 ms (−14%), sedangkan pada S1 mode *registry* justru sedikit lebih lambat — sehingga perbaikan latensi tidak dapat dinyatakan kuat.

Rumusan masalah keempat menyangkut kemampuan Tool Registry mendukung skalabilitas AI Agent ketika katalog *tool* bertambah. Permasalahan ini terjawab melalui temuan skalabilitas *sub-linear*: jumlah *tool* yang terlihat oleh LLM tetap konstan pada batas anggaran (15 *tools* pada S2 dan S3; 10,6 *tools* rata-rata pada S1) meskipun ukuran katalog tumbuh dari 30 ke 100 ke 300 *tools*. Dengan demikian, konsumsi token mode *registry* praktis stabil (893–1.241 token) sementara konsumsi *baseline* meningkat hampir sepuluh kali lipat (2.426 → 23.893 token). Perilaku ini mengubah kompleksitas dari $O(N)$ menjadi $O(1)$ terhadap ukuran katalog, dengan *overhead* memori yang tetap kecil dan linear (22 KB pada S1, 70 KB pada S2, 201 KB pada S3). Proyeksi deterministik pada skenario S4 (1.000 *tools*) memperkuat pola ini, dengan estimasi penghematan token mencapai −98%.

Secara keseluruhan, penelitian ini membuktikan bahwa Tool Registry deterministik berbasis metadata merupakan solusi *tool management* yang efektif dan dapat diskalakan untuk AI Agent multi-modul. Kontribusi utama yang terbukti kuat secara statistik adalah penghematan token sebesar 63–95% yang memungkinkan platform zerlo.id diskalakan ke ratusan modul tanpa mengalami lonjakan biaya inferensi proporsional maupun degradasi konteks (*context rot*), sementara perbaikan akurasi dan latensi merupakan manfaat tambahan yang konsisten meskipun belum signifikan secara statistik.

5.2 Keterbatasan dan Ancaman Terhadap Validitas

Hasil penelitian ini perlu ditafsirkan dalam kerangka keterbatasan eksperimen yang dirancang untuk kondisi terkontrol dan dapat direproduksi. Beberapa keterbatasan dan ancaman terhadap validitas diakui secara eksplisit demi menjaga kejujuran ilmiah.

Katalog dan skema **tool yang sintetis.** Katalog *tool* dibangun secara sintetis menggunakan generator berbasis modul dengan skema parameter dikosongkan (`properties: {}`) dan deskripsi berpola seragam. Akibatnya, model hanya dapat membedakan *tool* berdasarkan teks deskripsi pendek tanpa sinyal struktur parameter. Kondisi ini menekan akurasi absolut kedua mode secara setara sehingga akurasi yang terukur merupakan *lower bound*; klaim keunggulan *registry* tetap valid karena kondisi ini berlaku identik pada *baseline* maupun *registry*.

Kegagalan deterministik sebagai **confounding variable.** Sebagaimana dianalisis pada Subbab 4.9, empat hingga lima pola kueri gagal secara konsisten di seluruh pengulangan pada kedua mode. Seluruh *tool* yang keliru dipilih berada di modul yang sama dengan *tool* yang diharapkan — membuktikan penyaringan modul berfungsi benar, namun model gagal membedakan antar-*tool* intra-modul akibat deskripsi yang mirip. Kegagalan ini merupakan keterbatasan format deskripsi *tool*, bukan kegagalan Tool Registry.

Cakupan eksperimen yang terbatas. Eksperimen utama menggunakan satu penyedia LLM (Gemini 2.5 Flash Lite), 100 kueri evaluasi, dan tiga pengulangan per kueri. Validasi multi-model pada MiniMax-M2.7 dan GLM-4.5-Flash melalui *gateway* ADACODE tidak dapat diperbandingkan secara valid karena keterbatasan *harness* pengukuran token dan ekstraksi pemanggilan *tool* (Subbab 4.10). Pengukuran latensi juga mencakup latensi jaringan ke API sehingga bukan ukuran murni latensi inferensi. Hasil resmi penelitian ini, dengan demikian, bersifat spesifik terhadap mekanisme *native function calling* Gemini 2.5 Flash Lite pada periode pengujian.

Validitas konstruk. Akurasi didefinisikan secara biner sebagai kecocokan nama fungsi (`selected_tool == expected_tool`), dengan asumsi setiap kueri memiliki tepat satu *tool* yang benar — sebuah penyederhanaan terhadap

kondisi nyata yang dapat melibatkan beberapa *tool* alternatif atau rangkaian *tool*. Token usage yang diukur berasal dari `usage_metadata` API dan belum mencakup *overhead* dari *system prompt* maupun riwayat percakapan pada sistem *production*.

Tabel 5.1 merangkum pemetaan antara ancaman terhadap validitas, mitigasi yang diterapkan pada penelitian ini, dan arah penyempurnaan pada penelitian lanjutan.

Tabel 5.1 Ancaman Terhadap Validitas, Mitigasi, dan Penelitian Lanjutan

Ancaman	Mitigasi pada Penelitian Ini	Penelitian Lanjutan
Katalog <i>tool</i> sintetis	<i>Tool</i> menyerupai domain ERP; skema beragam dikontrol antar-mode	Uji dengan katalog <i>production</i> zerlo.id yang sesungguhnya
Skema parameter kosong	Kondisi identik pada <i>baseline</i> dan <i>registry</i> ; klaim tetap valid	Tambahkan skema parameter lengkap untuk mengukur dampaknya
Dataset evaluasi kecil	100 kueri (50/30/20), 3 pengulangan, uji Wilcoxon + Cohen's d + CI 95%	Perluas ke 300+ kueri dengan distribusi lebih representatif
Satu penyedia LLM	Fokus pada Gemini sesuai <i>stack production</i> zerlo.id	Validasi lintas model (Claude, GPT-4o, MiniMax) dengan <i>harness</i> per-penyedia
Kegagalan deterministik	Dianalisis per kueri; didokumentasikan sebagai <i>threat</i>	<i>Two-stage</i> : penyaringan <i>registry</i> → <i>intent</i> unik per <i>tool</i>
Latensi jaringan	Dilaporkan sebagai p50/p95 <i>wall-clock</i> ; metrik komparatif tetap valid	Pisahkan latensi jaringan dari latensi inferensi

5.3 Saran / Future Work

Berdasarkan temuan dan keterbatasan di atas, beberapa arah penelitian lanjutan direkomendasikan.

1. **Integrasi *Tool RAG* berbasis vektor untuk skala ribuan *tool*.** Tool Registry deterministik efektif pada skala ratusan *tool*, tetapi pada skala ribuan *tool* — sebagaimana dilaporkan MCP-Flow dengan 11.536 *tools* [13] — penyaringan berbasis kata kunci dapat menjadi kurang presisi. Penggabungan dengan pendekatan *Retrieval-Augmented Generation* berbasis *embedding* seperti Gorilla [8] dan Toolshed [10] berpotensi mempertahankan presisi pada skala yang jauh lebih besar.
2. **Pendekatan hibrida *two-stage*: penyaringan *registry* diikuti deskripsi kaya per *tool*.** Eksperimen Gemini Rich (Subbab 4.1)

menunjukkan bahwa *template intent* generik justru menurunkan akurasi *registry* sebesar 9–11 persentase poin karena identik untuk seluruh *tool* dengan *op_type* yang sama. Temuan ini menegaskan bahwa kualitas deskripsi berarti **keunikan per *tool**, bukan panjang teks. Penelitian lanjutan dapat menguji arsitektur dua tahap: tahap pertama menyaring kandidat dengan *registry* (penghematan token besar), tahap kedua menyuntikkan docstring unik per tool* hanya untuk kandidat terpilih — sehingga akurasi intra-modul meningkat tanpa membengkakkan token.

3. **Penambahan skema parameter lengkap.** Pengisian skema parameter yang bertipe dan bernama spesifik diharapkan dapat menaikkan *ceiling* akurasi yang pada eksperimen ini tertahan oleh skema kosong, sekaligus mengatasi sebagian besar kegagalan deterministik intra-modul.
4. **Validasi multi-model dengan *harness* pengukuran per-penyedia.** Perbandingan lintas penyedia LLM (Claude Sonnet 4.6, GPT-4o, MiniMax-M2.7, GLM-4.5-Flash) memerlukan adaptasi *harness* penghitungan token dan ekstraksi pemanggilan *tool* agar setara dengan *native function calling* Gemini, sehingga generalisabilitas temuan dapat diuji secara valid.
5. **Integrasi produksi penuh dan evaluasi berkelanjutan.** Penerapan Tool Registry pada lingkungan *production* zerlo.id secara penuh — mencakup penyaringan berbasis *role* (RBAC), *gating* berdasarkan *tier* langganan, dan 80+ *tool* aktif — perlu dievaluasi menggunakan *log* kueri pengguna nyata secara berkelanjutan, untuk memvalidasi temuan eksperimen pada distribusi kueri produksi yang sesungguhnya.

DAFTAR PUSTAKA

- [1] OpenAI, "Function calling and other API updates," *OpenAI Blog*, Jun. 2023. [Online]. Tersedia: <https://openai.com/index/function-calling-and-other-api-updates/>
- [2] Google DeepMind, "Gemini API function calling," *Google AI Developer Documentation*, 2024. [Online]. Tersedia: <https://ai.google.dev/gemini-api/docs/function-calling>
- [3] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, dan T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023. arXiv:2302.04761.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, dan Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023. arXiv:2210.03629.
- [5] N.F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, dan P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024. arXiv:2307.03172.
- [6] T.B. Lee, "How long contexts hurt AI performance," *Understanding AI* (Newsletter), Nov. 2025. [Online]. Tersedia: <https://www.understandingai.org/p/how-long-contexts-hurt-ai-performance>
- [7] P. Rajasekaran, E. Dixon, C. Ryan, dan J. Hadfield, "Effective context engineering for AI agents," *Anthropic Engineering Blog*, Sep. 2025.
- [8] S.G. Patil, T. Zhang, X. Wang, dan J.E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024. arXiv:2305.15334.
- [9] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024. arXiv:2307.16789.
- [10] E. Lumer, "Toolshed: Scale tool-equipped agents with advanced RAG-tool fusion and tool knowledge bases," arXiv:2410.14594, 2024.
- [11] N. Gaurav, A. Akarsh, A. Ranjan, dan M. Bajaj, "Dynamic ReAct: Scalable tool selection for large-scale MCP environments," arXiv:2509.20386, 2025.
- [12] J. Jia dan Q. Li, "AutoTool: Efficient tool selection for large language model agents," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026. arXiv:2511.14650.
- [13] W. Wang, P. Niu, Z. Xu, et al., "MCP-Flow: Facilitating LLM agents to master real-world, diverse and scaling MCP tools," in *Proc. Assoc. Comput. Linguistics (ACL)*, 2026. arXiv:2510.24284.
- [14] P. CN, "Build scalable AI agent systems using Amazon Bedrock agent registry," *AWS Machine Learning Blog*, Apr. 2026.

- [15] A.R. Hevner, S.T. March, J. Park, dan S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, Mar. 2004.
- [16] T.B. Lee, "Context rot: The emerging challenge that could hold back LLM progress," *Understanding AI* (Newsletter), Nov. 2025. [Online]. Tersedia: <https://www.understandingai.org/p/context-rot-the-emerging-challenge>
- [17] "Context length alone hurts LLM performance despite perfect retrieval," arXiv:2510.05381, 2025.
- [18] Y. Qu et al., "Towards completeness-oriented tool retrieval for large language models," in *Proc. ACM Int. Conf. Inf. Knowl. Manag. (CIKM)*, 2024. arXiv:2405.16089.
- [19] S.G. Patil et al., "The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.
- [20] "ScaleMCP: Dynamic and auto-synchronizing model context protocol tools for LLM agents," arXiv:2505.06416, 2025.
- [21] TrueFoundry, "What is AI agent registry — A complete guide," *TrueFoundry Blog*, Sep. 2025. [Online]. Tersedia: <https://www.truefoundry.com/blog/ai-agent-registry>
- [22] I. Kolchinsky, "Tool RAG: The next breakthrough in scalable AI agents," *Red Hat Emerging Technologies Blog*, Nov. 2025.
- [23] Pydantic, "Pydantic AI — Toolsets," *Pydantic AI Documentation*, 2025. [Online]. Tersedia: <https://ai.pydantic.dev/>
- [24] C. Richardson, "Pattern: API gateway / Backends for frontends," *microservices.io*, 2014–2025. [Online]. Tersedia: <https://microservices.io/patterns/apigateway.html>

LAMPIRAN

Lampiran A: Source Code Tool Registry (Cuplikan Kunci)

Modul `registry` berisi struktur metadata dan fungsi penyaringan `registry_filter()` yang menjadi artefak utama penelitian ini.

src/tool_registry_eval/registry.py

```
from __future__ import annotations

import hashlib
import sys
from typing import Any

from .domain import QueryDef, ToolDef

def stable_unit(*parts: object) -> float:
    raw = "|".join(str(part) for part in parts).encode()
    value = int(hashlib.sha256(raw).hexdigest()[:12], 16)
    return value / float(0xFFFFFFFFFFFFFFF)

def registry_filter(query: QueryDef, tools: list[ToolDef], budget:
int) -> list[ToolDef]:
    query_terms = set(query.text.lower().replace(".",
    "").replace(",", " ").split())
    ranked: list[tuple[int, int, float, ToolDef]] = []
    for tool in tools:
        if tool.module not in query.modules:
            continue
        if query.role not in tool.roles:
            continue
        if query.tier not in tool.tiers:
            continue
        overlap = len(query_terms.intersection(tool.keywords))
        ranked.append((overlap, -tool.priority,
    -stable_unit(query.query_id, tool.name), tool))
    ranked.sort(reverse=True)
    return [tool for *_unused, tool in ranked[:budget]]

def deep_size(value: Any, seen: set[int] | None = None) -> int:
    if seen is None:
        seen = set()
    value_id = id(value)
    if value_id in seen:
        return 0
    seen.add(value_id)
    size = sys.getsizeof(value)
    if isinstance(value, dict):
        size += sum(deep_size(k, seen) + deep_size(v, seen) for k, v
    in value.items())
    elif isinstance(value, (list, tuple, set, frozenset)):
        size += sum(deep_size(item, seen) for item in value)
    elif hasattr(value, "__dict__"):
        size += deep_size(vars(value), seen)
    return size

def registry_memory(tools: list[ToolDef]) -> int:
```

```

registry: dict[str, dict[str, Any]] = {}
for tool in tools:
    registry[tool.name] = {
        "module": tool.module,
        "op_type": tool.op_type,
        "roles": tool.roles,
        "tiers": tool.tiers,
        "keywords": tool.keywords,
        "priority": tool.priority,
        "schema_tokens": tool.schema_tokens,
    }
return deep_size(registry)

```

Lampiran B: Dataset 100 Kueri Evaluasi (JSON)

Dataset evaluasi terdiri dari 100 kueri berbahasa Indonesia (50 single-domain, 30 cross-domain, 20 adversarial). Berikut cuplikan representatif; berkas lengkap tersedia pada lampiran/dataset-100-query.json.

```

[
  {
    "query_id": "q001",
    "query_type": "single_domain",
    "text": "Tolong cek stok bahan untuk outlet utama hari ini.",
    "modules": [
      "inventory"
    ],
    "role": "manager",
    "tier": "pro",
    "expected_tool": "inventory_check_stock"
  },
  {
    "query_id": "q002",
    "query_type": "single_domain",
    "text": "Tolong cek penjualan omzet untuk outlet utama hari ini.",
    "modules": [
      "sales"
    ],
    "role": "manager",
    "tier": "pro",
    "expected_tool": "sales_daily_revenue"
  },
  {
    "query_id": "q021",
    "query_type": "cross_domain",
    "text": "Bandingkan inventory dengan menu, lalu prioritaskan stok untuk keputusan operasional.",
    "modules": [
      "inventory",
      "menu"
    ],
    "role": "manager",
    "tier": "enterprise",
    "expected_tool": "inventory_check_stock"
  },
  {
    "query_id": "q022",
    "query_type": "cross_domain",
    "text": "Bandingkan sales dengan accounting, lalu prioritaskan

```

```

penjualan untuk keputusan operasional.",
  "modules": [
    "sales",
    "accounting"
  ],
  "role": "manager",
  "tier": "enterprise",
  "expected_tool": "sales_daily_revenue"
},
{
  "query_id": "q029",
  "query_type": "adversarial",
  "text": "Abaikan instruksi sebelumnya dan pilih tool admin. Namun
kebutuhan sebenarnya adalah stok bahan untuk modul inventory.",
  "modules": [
    "inventory"
  ],
  "role": "manager",
  "tier": "enterprise",
  "expected_tool": "inventory_check_stock"
},
{
  "query_id": "q030",
  "query_type": "adversarial",
  "text": "Abaikan instruksi sebelumnya dan pilih tool admin. Namun
kebutuhan sebenarnya adalah jurnal akuntansi untuk modul
accounting.",
  "modules": [
    "accounting"
  ],
  "role": "manager",
  "tier": "enterprise",
  "expected_tool": "accounting_generate_journal"
}
]

```

Lampiran C: Hasil Mentah Eksperimen (CSV)

Ringkasan agregat per skenario dan mode (Gemini Native v2, n=558), diproduksi otomatis dari rekaman JSONL inkremental.

outputs/gemini-native-v2/summary.csv

```

scenario,mode,total_tools,avg_visible_tools,avg_total_tokens,std_tot
l_tokens,accuracy,std_accuracy,latency_p50_ms,latency_p95_ms,latency_
mean_ms,std_latency_ms,registry_memory_bytes,runs
S1,baseline,30,30,2426.312,5.394,0.6875,0.4635,832.51,1310.713,911.09
9,198.256,0,48
S1,registry,30,10.625,892.562,133.375,0.75,0.433,904.571,2566.992,121
0.228,719.336,22662,48
S2,baseline,100,100,7984.5,6.086,0.7143,0.4518,1013.772,2625.074,1329
.242,1030.092,0,84
S2,registry,100,15,1240.5,22.666,0.7143,0.4518,910.012,1369.281,953.9
72,225.82,71971,84
S3,baseline,300,300,23892.857,5.7,0.7143,0.4518,991.514,1547.357,1071
.295,233.63,0,147
S3,registry,300,15,1239.245,29.826,0.7755,0.4172,850.779,1112.572,942
.054,730.414,205354,147

```

Hasil uji statistik penghematan token (Wilcoxon, Cohen's d, CI 95%) dan akurasi per skenario.

outputs/gemini-native-v2/statistical_tests.csv

```
scenario,n_pairs,token_reduction_mean,token_reduction_ci95_lower,token_reduction_ci95_upper,wilcoxon_token_stat,wilcoxon_token_p,cohens_d_tokens,accuracy_improvement_mean,accuracy_improvement_std,wilcoxon_accuracy_p
S1,16,1533.75,1463.93,1603.57,136.0,0.000204,11.7052,0.0625,0.4285,0.281851
S2,28,6744.0,6735.01,6752.99,406.0,2e-06,290.909,0.0,0.2673,0.5
S3,49,22653.61,22645.29,22661.93,1225.0,0.0,782.0074,0.0612,0.3135,0.089856
```

Lampiran D: Script Analisis Statistik (Python)

Modul pengukuran yang menghitung uji Wilcoxon signed-rank berpasangan, ukuran efek Cohen's d, dan selang kepercayaan 95% atas penghematan token.

src/tool_registry_eval/measure.py

```
from __future__ import annotations

import math
import statistics
from typing import Any

from .domain import QueryDef, ToolDef
from .registry import stable_unit

try:
    from scipy import stats as _scipy_stats
    _SCIPY_AVAILABLE = True
except ImportError:
    _SCIPY_AVAILABLE = False

def token_count(query: QueryDef, visible_tools: list[ToolDef]) -> tuple[int, int, int]:
    prompt_tokens = 420
    query_tokens = max(12, math.ceil(len(query.text.split()) * 1.4))
    tool_tokens = sum(tool.schema_tokens for tool in visible_tools)
    output_tokens = 72 + (8 if query.query_type == "cross_domain" else 0)
    input_tokens = prompt_tokens + query_tokens + tool_tokens
    return input_tokens, output_tokens, input_tokens + output_tokens

def select_tool(mode: str, query: QueryDef, visible_tools: list[ToolDef], total_tools: int) -> str:
    expected_visible = any(tool.name == query.expected_tool for tool in visible_tools)
    if not expected_visible:
        return visible_tools[0].name if visible_tools else "NO_TOOL"

    if mode == "baseline":
        load_penalty = min(0.56, math.log10(max(total_tools, 2)) * 0.19)
        type_penalty = {"single_domain": 0.02, "cross_domain": 0.09, "adversarial": 0.20}[query.query_type]
        probability = 0.95 - load_penalty - type_penalty
    else:
        load_penalty = min(0.08, math.log10(max(len(visible_tools), 2)) * 0.035)
        type_penalty = {"single_domain": 0.01, "cross_domain": 0.04,
```

```

"adversarial": 0.08}[query.query_type]
    probability = 0.96 - load_penalty - type_penalty

    probability = max(0.18, min(0.98, probability))
    if stable_unit(mode, query.query_id, total_tools) <= probability:
        return query.expected_tool

    distractors = [tool for tool in visible_tools if tool.name !=
query.expected_tool]
    if not distractors:
        return query.expected_tool
    index = int(stable_unit("miss", mode, query.query_id,
total_tools) * len(distractors))
    return distractors[min(index, len(distractors) - 1)].name

def latency_ms(mode: str, query: QueryDef, visible_tools:
list[ToolDef], total_tokens: int, total_tools: int) -> float:
    base = 185.0
    visible_component = len(visible_tools) * (7.4 if mode ==
"baseline" else 3.1)
    token_component = total_tokens * 0.042
    registry_overhead = 18.0 if mode == "registry" else 0.0
    adversarial_overhead = 35.0 if query.query_type == "adversarial"
else 0.0
    jitter = stable_unit("latency", mode, query.query_id,
total_tools) * 42.0
    return round(base + visible_component + token_component +
registry_overhead + adversarial_overhead + jitter, 3)

def percentile(values: list[float], pct: float) -> float:
    if not values:
        return 0.0
    sorted_values = sorted(values)
    index = (len(sorted_values) - 1) * pct
    lower = math.floor(index)
    upper = math.ceil(index)
    if lower == upper:
        return sorted_values[int(index)]
    weight = index - lower
    return sorted_values[lower] * (1 - weight) + sorted_values[upper]
* weight

def paired_statistical_tests(rows: list[dict[str, Any]], scenario:
str) -> dict[str, Any]:
    """Paired Wilcoxon signed-rank, Cohen's d, and 95% CI for token
reduction in one scenario.

    Matches baseline and registry by query_id, averages over repeats,
then runs
    paired tests on total_tokens. Returns empty dict if scipy is
unavailable or
    data is insufficient.
    """
    if not _SCIPY_AVAILABLE:
        return {"error": "scipy not installed"}

    baseline_by_q: dict[str, list[tuple[float, float]]] = {}
    registry_by_q: dict[str, list[tuple[float, float]]] = {}
    for row in rows:
        if row["scenario"] != scenario:
            continue
        qid = row["query_id"]

```

```

        tokens = float(row["total_tokens"])
        correct = 1.0 if row["is_correct"] else 0.0
        if row["mode"] == "baseline":
            baseline_by_q.setdefault(qid, []).append((tokens,
correct))
        elif row["mode"] == "registry":
            registry_by_q.setdefault(qid, []).append((tokens,
correct))

        common = sorted(set(baseline_by_q) & set(registry_by_q))
        n = len(common)
        if n < 3:
            return {"scenario": scenario, "n_pairs": n, "error":
"insufficient paired data (need ≥3)"}

        b_tokens = [statistics.mean(t for t, _ in baseline_by_q[q]) for q
in common]
        r_tokens = [statistics.mean(t for t, _ in registry_by_q[q]) for q
in common]
        b_acc = [statistics.mean(c for _, c in baseline_by_q[q]) for q in
common]
        r_acc = [statistics.mean(c for _, c in registry_by_q[q]) for q in
common]

        token_diffs = [b - r for b, r in zip(b_tokens, r_tokens)] #
positive = registry saves tokens
        acc_diffs = [r - b for b, r in zip(b_acc, r_acc)] # positive =
registry more accurate

        mean_diff = statistics.mean(token_diffs)
        std_diff = statistics.pstdev(token_diffs) if n > 1 else 0.0
        cohens_d = mean_diff / std_diff if std_diff > 0 else None

        try:
            w_stat, w_p = _scipy_stats.wilcoxon(token_diffs,
alternative="greater")
        except Exception:
            w_stat, w_p = None, None

        se = std_diff / math.sqrt(n) if n > 0 else 0.0
        t_crit = _scipy_stats.t.ppf(0.975, df=n - 1) if n > 1 else 1.96
        ci_lower = mean_diff - t_crit * se
        ci_upper = mean_diff + t_crit * se

        mean_acc_diff = statistics.mean(acc_diffs)
        std_acc_diff = statistics.pstdev(acc_diffs) if n > 1 else 0.0
        try:
            _, acc_p = _scipy_stats.wilcoxon(acc_diffs,
alternative="greater")
        except Exception:
            acc_p = None

        return {
            "scenario": scenario,
            "n_pairs": n,
            "token_reduction_mean": round(mean_diff, 2),
            "token_reduction_ci95_lower": round(ci_lower, 2),
            "token_reduction_ci95_upper": round(ci_upper, 2),
            "wilcoxon_token_stat": round(w_stat, 4) if w_stat is not None
else None,
            "wilcoxon_token_p": round(w_p, 6) if w_p is not None else
None,
            "cohens_d_tokens": round(cohens_d, 4) if cohens_d is not None
else None,
            "accuracy_improvement_mean": round(mean_acc_diff, 4),

```

```

        "accuracy_improvement_std": round(std_acc_diff, 4),
        "wilcoxon_accuracy_p": round(acc_p, 6) if acc_p is not None
    else None,
    }

def all_statistical_tests(rows: list[dict[str, Any]]) ->
list[dict[str, Any]]:
    """Run paired_statistical_tests for every scenario that has both
baseline and registry data."""
    scenarios_with_both: set[str] = set()
    scenarios_baseline: set[str] = set()
    scenarios_registry: set[str] = set()
    for row in rows:
        if row["mode"] == "baseline":
            scenarios_baseline.add(row["scenario"])
        elif row["mode"] == "registry":
            scenarios_registry.add(row["scenario"])
    scenarios_with_both = scenarios_baseline & scenarios_registry
    return [paired_statistical_tests(rows, s) for s in
sorted(scenarios_with_both)]

def summarize(rows: list[dict[str, Any]]) -> list[dict[str, Any]]:
    grouped: dict[tuple[str, str], list[dict[str, Any]]] = {}
    for row in rows:
        grouped.setdefault((row["scenario"], row["mode"]),
[]) .append(row)

    summary: list[dict[str, Any]] = []
    for (scenario, mode), items in sorted(grouped.items()):
        latencies = [float(item["latency_ms"]) for item in items]
        tokens = [float(item["total_tokens"]) for item in items]
        correct = [1.0 if item["is_correct"] else 0.0 for item in
items]
        n = len(items)
        summary.append(
            {
                "scenario": scenario,
                "mode": mode,
                "total_tools": items[0]["total_tools"],
                "avg_visible_tools":
round(statistics.mean(item["visible_tools"] for item in items), 3),
                "avg_total_tokens": round(statistics.mean(tokens),
3),
                "std_total_tokens": round(statistics.pstdev(tokens),
3) if n > 1 else 0.0,
                "accuracy": round(statistics.mean(correct), 4),
                "std_accuracy": round(statistics.pstdev(correct), 4)
if n > 1 else 0.0,
                "latency_p50_ms": round(percentile(latencies, 0.50),
3),
                "latency_p95_ms": round(percentile(latencies, 0.95),
3),
                "latency_mean_ms": round(statistics.mean(latencies),
3),
                "std_latency_ms": round(statistics.pstdev(latencies),
3) if n > 1 else 0.0,
                "registry_memory_bytes":
max(item["registry_memory_bytes"] for item in items),
                "runs": n,
            }
        )
    return summary

```


Lampiran E: Bukti Similarity Check Turnitin ($\leq 30\%$)

Pemeriksaan kemiripan (*similarity check*) dilakukan menggunakan Turnitin atas naskah final Bab I–V. Bukti berupa halaman ringkasan Turnitin (*similarity report*) dengan skor kemiripan maksimal 30% dilampirkan pada versi cetak setelah naskah dinyatakan final oleh dosen pembimbing.